DOCUMENTO TÉCNICO DEL ECOSISTEMA

Culinary

Funcionalidades, procesos, estructura, lenguajes, workflows, metodologías y líneas de código

Culinary Restaurant	repositorio MesaNexoNativo · punto de venta y gestión del restaurante
CulinaryOS	repositorio CulinaryOS · sistema operativo Debian 13 de la sucursal
Culinary Admin	repositorio CulinaryAdmin · panel del proveedor
Culinary Server	repositorio CulinaryServer · Cloudflare Worker de licencias y versiones

22,647
líneas de producción

9,118
líneas de pruebas

895
archivos versionados

99
commits sin merges

Estado del código al 31 de octubre de 2026 · versión funcional 0.3.0 · Medido sobre los commits de git de cada rama

Índice

1. Resumen 4

Lo esencial . . . 4

Cómo leer este documento . . . 4

2. Arquitectura del ecosistema 5

Reglas de diseño . . . 5

Protocolos entre componentes . . . 6

3. Líneas de código 7

Cómo se midió . . . 7

Lenguajes . . . 8

Componentes de Culinary Restaurant por tamaño . . . 9

Ritmo de trabajo . . . 9

4. Lenguajes y tecnologías 10

Por qué cada lenguaje . . . 10

5. Culinary Restaurant 11

5.1 Estructura . . . 11

5.2 Flujo de una mutación . . . 11

5.3 Roles y permisos . . . 12

5.4 Funcionalidades . . . 13

5.5 Backend: rutas y workers . . . 13

5.6 Almacén y cifrado . . . 14

5.7 Outbox y sincronización . . . 14

5.8 Bitácora y puntos de control . . . 14

5.9 Licencias en la sucursal . . . 15

5.10 Integraciones . . . 15

5.11 Cliente y kiosco . . . 15

5.12 Central y portal público . . . 16

5.13 Reglas de dinero . . . 16

6. CulinaryOS 17

6.1 Estructura . . . 17

6.2 Módulos . . . 17

6.3 Perfiles . . . 18

6.4 Instalación y primer inicio . . . 18

6.5 Servicios . . . 18

6.6 Red . . . 18

6.7 Seguridad y cifrado . . . 19

6.8 Respaldo y recuperación . . . 19

6.9 UPS y operación física . . . 19

7. Culinary Server 20

7.1 Estructura . . . 20

7.2 Modelo de datos . . . 20

7.3 Cadena de llaves . . . 21

7.4 Solicitudes firmadas . . . 21
7.5 Reglas de licencia . . . 21
7.6 Herramientas . . . 22

8. Culinary Admin **22**
8.1 Funcionalidades . . . 22
8.2 Seguridad y diseño . . . 22
8.3 Compilación . . . 22

9. Workflows de punta a punta **23**
9.1 Vender en el comedor . . . 23
9.2 Cobro con terminal y resultado incierto . . . 23
9.3 Activar y renovar una licencia . . . 23
9.4 Publicar una actualización . . . 23
9.5 Sustituir un servidor dañado . . . 24
9.6 Sin Internet o sin central . . . 24
9.7 Construir la ISO . . . 24
9.8 Instalar la aplicación en un CulinaryOS existente . . . 24

10. Ramas, una por una **25**
10.1 Culinary Restaurant . . . 25
10.2 CulinaryOS . . . 26
10.3 Culinary Admin y Culinary Server . . . 27

11. Metodologías de trabajo **27**
Patrones de arquitectura . . . 28
Convenciones . . . 28

12. Pruebas y CI **28**
Cobertura por riesgo . . . 29
Integración continua . . . 29
Carga . . . 29

13. Estado real y riesgos **30**
Auditoría 0.3.0 . . . 30
Antes de vender . . . 30
Riesgos funcionales abiertos . . . 31
Discrepancias encontradas al revisar . . . 31

Anexo A. Archivos y líneas **32**

Anexo B. Rutas de API **36**
Backend de sucursal, MesaNexo.Api . . . 36
MesaNexo.Central y MesaNexo.Public . . . 36
Culinary Server . . . 37

Anexo C. Acciones del dominio **38**

Anexo D. Glosario **39**

1. Resumen

Culinary es una plataforma para operar restaurantes en México. Se compone de cuatro productos en cuatro repositorios. El restaurante trabaja sobre un servidor propio en su local, sin depender de Internet. La nube sólo se usa para licencias, actualizaciones, respaldo externo y canales opcionales como pedidos en línea.

Producto	Qué es	Dónde corre	Lenguaje principal	Producción	Pruebas
Culinary Restaurant	Punto de venta, cocina, caja, inventario, administración, integraciones, central multiempresa y portal público	Sucursal, terminales y nube opcional	C#, TypeScript	16,398	6,701
CulinaryOS	Sistema operativo de la sucursal, instalador, kiosco, red, certificados, respaldo y diagnóstico	Equipo servidor de la sucursal	Python, Bash	3,639	1,420
Culinary Server	API de licencias, versiones, soporte y registro de sucursales	Cloudflare Workers	TypeScript	1,660	756
Culinary Admin	Panel del proveedor para operar esa API	Windows y Android	TypeScript, React	950	241
Total				22,647	9,118

Lo esencial

- **La sucursal manda.** El backend de cada restaurante es la única autoridad de mesas, cuentas, cobros, caja, inventario y permisos. Los clientes guardan sólo interfaz, conexión y solicitudes pendientes.
- **Una venta no depende de la red externa.** Cada mutación se confirma en PostgreSQL local con su recibo, auditoría y evento de salida en la misma transacción. Después, un worker la envía a la central si existe.
- **Nada se duplica por un corte.** Cada operación lleva un UUID y una huella de su contenido. Si se pierde la respuesta, el cliente consulta el recibo en vez de repetir el cobro.
- **Los datos van sellados.** Cada registro y evento se cifra con AES-256-GCM antes de escribirse, así que un volcado de la base no contiene datos legibles.
- **La licencia se firma fuera de línea.** Una raíz Ed25519 que nunca toca el Worker firma dos intermedias: una para licencias y otra para paquetes. Comprometer el Worker no permite autorizar bytes que la estación no firmó.
- **Hoy nada está validado en real ni habilitado para vender.** El código está hecho y pasa sus pruebas automáticas, con simuladores para los servicios externos. Falta el piloto (sección 13).

Cómo leer este documento

Las secciones 2 a 4 dan el panorama: arquitectura, métricas y lenguajes. Las secciones 5 a 8 describen cada producto. La 9 sigue los procesos de punta a punta. La 10 revisa cada rama por separado. Las secciones 11 a 13 cubren metodología, pruebas y estado real. Los anexos listan cada archivo con sus líneas, todas las rutas de API y todas las acciones del dominio.

Nombres. `MesaNexo` es el nombre técnico histórico de Culinary Restaurant: se conserva en proyectos, identificadores de instalación y rutas para no romper compatibilidad. En este documento significan lo mismo.

2. Arquitectura del ecosistema

El diagrama muestra qué habla con qué. Las flechas continuas son operación normal. Las punteadas sólo existen si el dueño configuró ese servicio.

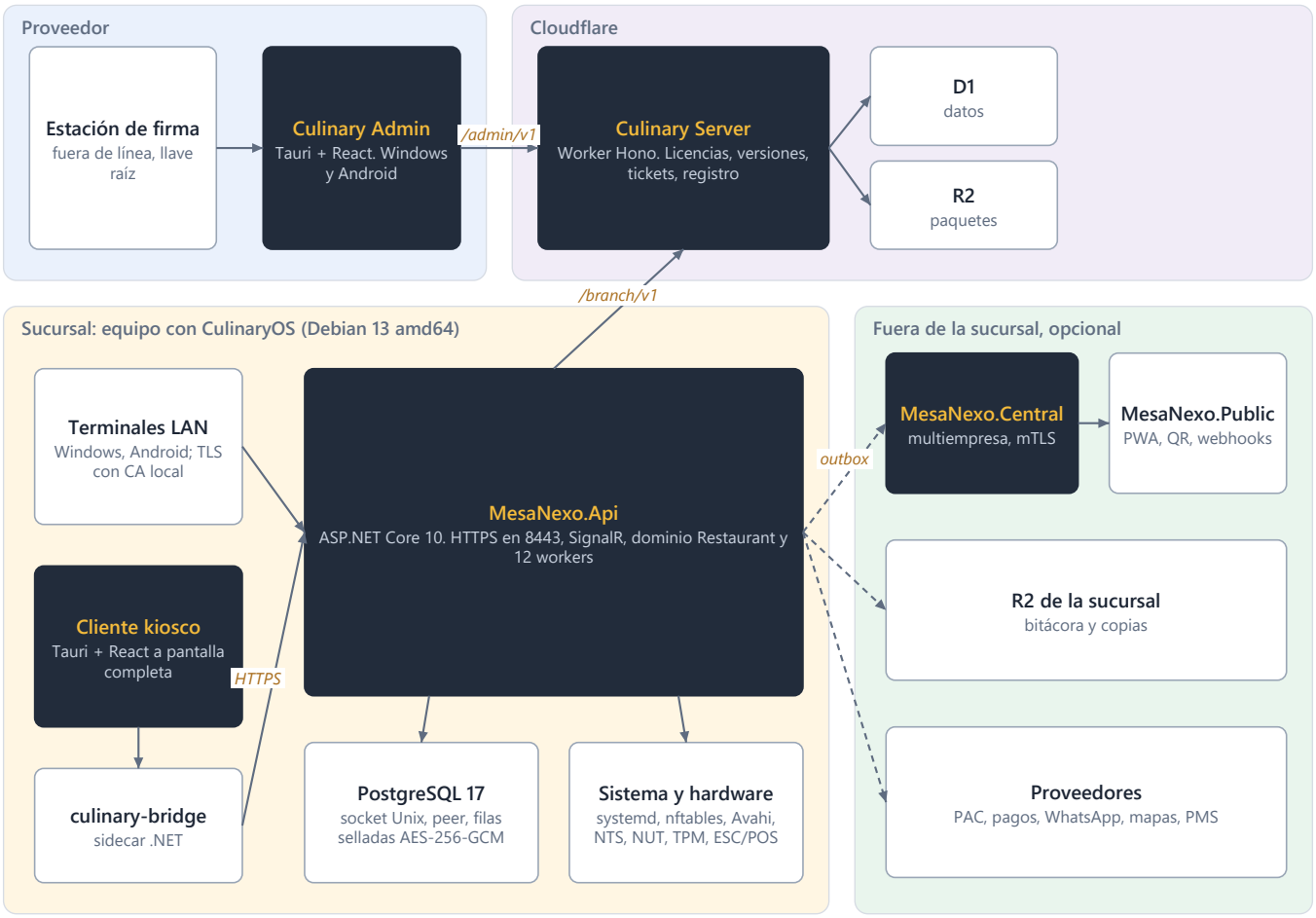


Figura 1. Componentes y conexiones. El backend usa sólo conexiones salientes hacia la nube: la sucursal no abre puertos hacia Internet.

Reglas de diseño

Regla	Cómo se cumple
Autoridad local	`BranchService` y `Restaurant` deciden. La central mantiene espejo y contratación; nunca reemplaza a la sucursal.
Operación sin Internet	Las rutas locales no consultan la central. La disponibilidad de LAN, central e Internet se muestra por separado.
Confianza explícita	TLS con CA aprovisionada, nombre y huella. No existe opción de aceptar cualquier certificado. mDNS sólo propone direcciones.
Un solo remitente	`CentralSyncWorker` es el único emisor de outbox. La política, los pedidos en la nube y las licencias van por canales separados.
Clientes sin secretos	Ninguna terminal recibe el llavero de datos ni credenciales de proveedores. Las credenciales viven en `/etc/culinaryos/integrations`.
Estados inciertos honestos	Un cobro, timbre o impresión sin respuesta queda «incierto» y requiere conciliar. Nunca se muestra como éxito.
Separación de proveedor	Los módulos de proveedor salieron de Culinary Restaurant. Viven en Culinary Admin y Culinary Server.

Protocolos entre componentes

De → a	Transporte	Autenticación	Contenido
React → Tauri	IPC `native_call`	Sólo ventana `main`	Comando y argumentos JSON
Tauri → culinary-bridge	stdin/stdout del proceso hijo	Binario fijo junto al ejecutable	JSON por línea, con id de secuencia, máximo 2 MB
Bridge/Terminal → API	HTTPS y SignalR `/live`	CA local, sesión y credencial de dispositivo	`/api/*`, avisos de revisión
API → PostgreSQL	Socket Unix	peer como `culinaryos-app`	Registros sellados por entidad
API → Central	HTTPS `/v1/events`, `/v1/cloud/*`	mTLS	Protocolo `culinaryos-events-v1`, FIFO, acuse tras commit
API → Server	HTTPS `/branch/v1`	Reto firmado con la llave de la licencia	Activación, renovación, actualizaciones, tickets
Admin → Server	HTTPS `/admin/v1`	Llave de dispositivo, contraseña, TOTP, sesión	Licencias, versiones, soporte, registro
API → R2	HTTPS firmada	Credenciales propias por sucursal	Puntos de control y copias `pg_dump`

3. Líneas de código

El ecosistema tiene **22,647 líneas de producción** y **9,118 líneas de pruebas** en sus ramas de referencia, una proporción de 0.40 líneas de prueba por cada línea de producción. La documentación suma 4,276 líneas de Markdown.

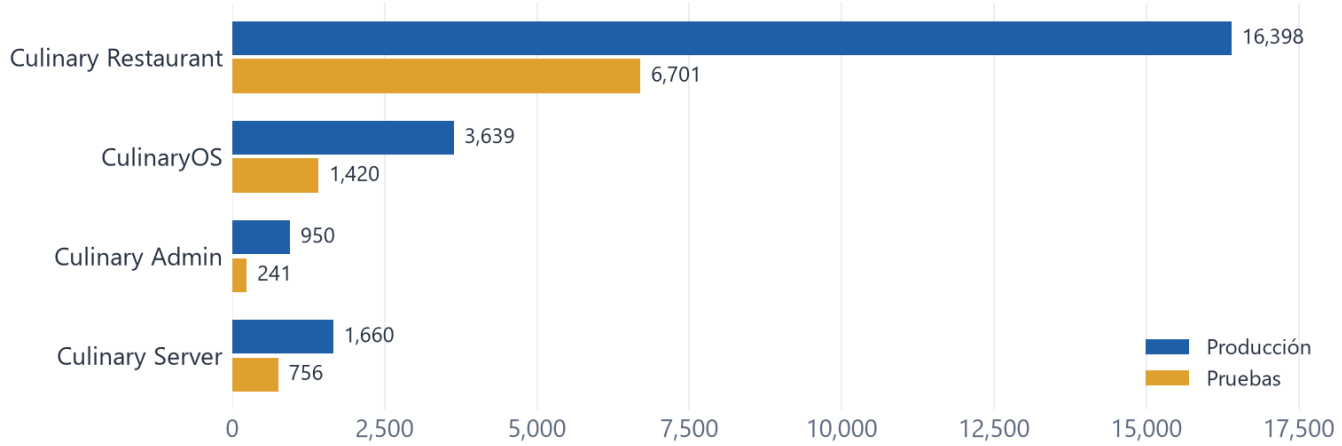


Figura 2. Líneas de código sin blancos ni comentarios, en la rama de referencia de cada repositorio.

Cómo se midió

- Se leyeron los blobs de git de cada rama, no la carpeta de trabajo. Los cambios sin confirmar no cuentan.
- **Producción** y **pruebas** son líneas con código: se excluyen líneas en blanco y comentarios detectados por prefijo. Es una medida aproximada: no separa comentarios al final de una línea.
- Se clasifica como prueba lo que está en `tests/`, `test/`, proyectos `.Tests` o archivos `.spec`.
- No se cuentan binarios, `node_modules`, `dist`, `bin`, `obj`, `target`, `gen`, `vendor`, archivos de bloqueo (`package-lock.json`, `Cargo.lock`, `packages.lock.json`), evidencia ni maquetas de diseño. Tampoco JSON de datos, SVG ni Markdown, que se reportan aparte.
- Ramas de referencia: `postgresql` en Culinary Restaurant y CulinaryOS, `master` en Culinary Admin, `main` en Culinary Server.

Repositorio	Rama	Commit	Archivos	Fuente	Producción	Pruebas	Docs	Datos	Bytes prod.
Culinary Restaurant	postgresql	d8fc1ca	591	269	16,398	6,701	1,719	10,557	1,699,398
CulinaryOS	postgresql	c753621	176	85	3,639	1,420	1,750	485	201,741
Culinary Admin	master	329bbcd	87	21	950	241	141	176	106,781
Culinary Server	main	d627abf	41	34	1,660	756	666	65	101,089
Total			895	409	22,647	9,118	4,276	11,283	2,109,009

Fuente: archivos de lenguaje de programación o configuración. Docs: Markdown y texto. Datos: JSON y SVG, incluidos los resultados de evidencia.

> **Lectura correcta.** El conteo subestima el tamaño real de partes densas del código. Hay 2,109,009 bytes de producción, unos 77 caracteres por línea en promedio, con métodos de dominio escritos en una sola línea larga. `src/MesaNexo.Core/Restaurant.cs` tiene 617 líneas, pero concentra gran parte de las reglas. Para comparar tamaño conviene mirar también los bytes.

Lenguajes

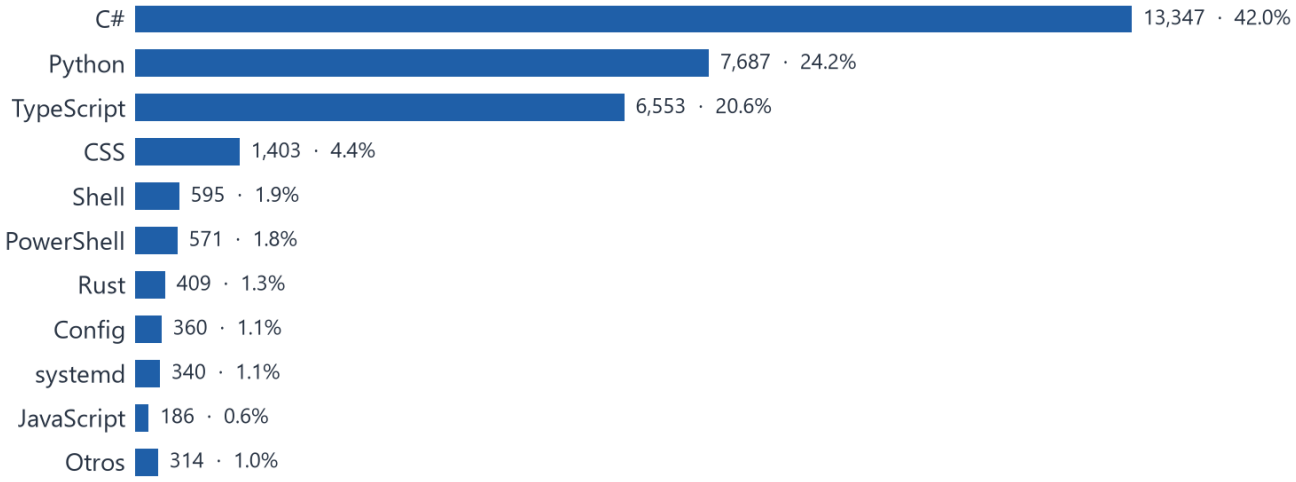


Figura 3. Líneas de código por lenguaje, producción y pruebas juntas, 31,765 en total. TypeScript incluye TSX. Otros agrupa HTML, SQL, YAML, XML, AXAML y TOML.

Lenguaje	Culinary Restaurant	CulinaryOS	Culinary Admin	Culinary Server	Total
C#	13,347	–	–	–	13,347
Python	3,812	3,875	–	–	7,687
TypeScript	1,266	–	388	2,168	3,822
TypeScript (TSX)	2,265	–	466	–	2,731
CSS	1,174	–	229	–	1,403
Shell	62	533	–	–	595
PowerShell	466	41	64	–	571
Rust	398	–	11	–	409
Config	54	292	9	5	360
systemd	22	318	–	–	340
JavaScript	63	–	–	123	186
SQL	–	–	–	120	120
YAML	57	–	–	–	57
XML (proyectos .NET)	41	–	–	–	41
AXAML	32	–	–	–	32
XML	27	–	–	–	27
TOML	10	–	11	–	21
HTML	3	–	13	–	16

Componentes de Culinary Restaurant por tamaño

Componente	estado-en-memoria	integraciones-produccion	postgresql
src/MesaNexo.Web	3,409	3,596	4,110
src/MesaNexo.Core	1,411	2,894	3,181
src/MesaNexo.Api	713	1,986	2,930
src/MesaNexo.Client	1,052	1,651	1,695
deploy/culinaryos	850	1,032	1,235
tools	702	849	924
src/MesaNexo.Central	429	586	601
src/MesaNexo.Integrations	90	512	525
runtime	330	383	433
src/MesaNexo.Bridge	154	199	199
deploy	145	167	167
src/MesaNexo.Public	64	80	79
(raiz)	48	72	74
src/MesaNexo.Android	69	71	71
src/MesaNexo.Desktop	58	66	67
.github/workflows	0	0	57
src/MesaNexo.iOS	44	49	49
src/MesaNexo.Transport	1	1	1
Pruebas	2,429	4,821	6,701

Código de producción por carpeta de primer nivel. El detalle archivo por archivo está en el Anexo A.

Ritmo de trabajo

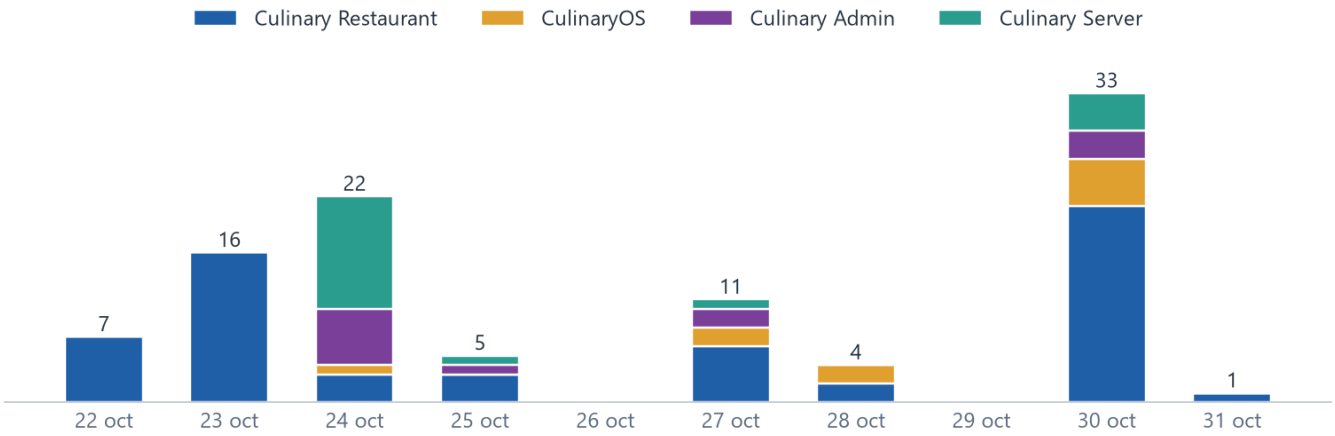


Figura 4. Commits sin merges por día en los cuatro repositorios: 99 en diez días, sobre un estado inicial que ya venía de trabajo anterior (el primer commit importa la versión 0.3.0-dev).

4. Lenguajes y tecnologías

Versiones tomadas de los archivos de proyecto (`*.csproj`, `global.json`, `package.json`, `Cargo.toml`, listas de paquetes de Debian).

Capa	Tecnología	Versión	Uso
Backend	C#, .NET, ASP.NET Core	SDK 10.0.402	API de sucursal, central, portal, puente y dominio
Tiempo real	SignalR	10.0.13	Aviso de revisión a clientes; el estado siempre se relee
Base de sucursal	PostgreSQL + Npgsql	17 (Debian) / 10.0.3	Almacén cifrado por registro, socket Unix
Base de central	PostgreSQL	misma pila	Multiempresa, espejos y pedidos
Cliente principal	React, TypeScript, CSS	19.1.1 / 5.9.3	Interfaz de operación, escritorio y kiosco
Empaquetado cliente	Vite, Tauri, Rust	7.3.7 / 2 / edición 2021	Ejecutable nativo con WebView2 o WebKitGTK
Cliente anterior y móvil	Avalonia	12.1.4	Se conserva durante la transición; base de Android e iOS
Criptografía .NET	BouncyCastle, Konscious Argon2	2.7.0 / 1.3.1	Firmas Ed25519 y contraseñas Argon2id
Correo y QR	MailKit, QRCoder	4.18.2 / 1.8.0	SMTP y códigos QR
Sistema operativo	Debian 13 trixie amd64	0.3.0-dev integrada	live-build, debian-installer, systemd
Runtime del SO	Python 3, PyNaCl	1.6.2 en laboratorio	Provisión, diagnóstico, PKI, respaldo, recuperación
Kiosco	LightDM, Openbox, WebKitGTK	de Debian 13	Sesión exclusiva sin escritorio
Red y hora	NetworkManager, nftables, Avahi, chrony NTS	de Debian 13	IP, firewall, mDNS y hora autenticada
Energía	NUT	de Debian 13	UPS USB; apagado en activación separada
Worker	TypeScript, Hono, Zod	5.9.3 / 4.13.13 / 4.6.7	API de licencias en Cloudflare Workers
Datos del Worker	Cloudflare D1 y R2	fecha de compatibilidad 2026-10-19	SQL y paquetes
Panel	React, Vite, Tauri, lucide-react, qrcode	19.1.0 / 7.1.5 / 2.8.4	Culinary Admin
Pruebas web	Playwright	1.56.0	Chromium y Edge
Pruebas del Worker	Vitest + plugin de Workers	5.0.5 / 1.4.7	Corren dentro del runtime de Workers
Automatización	PowerShell, Bash, GitHub Actions	–	Compilación, laboratorio, empaquetado, CI

Por qué cada lenguaje

- **C#** contiene el dominio. Comparte tipos entre backend, central y clientes, y se publica autocontenido para Linux y Windows.
- **TypeScript con React** concentra la interfaz. Se empaqueta dentro del ejecutable: no necesita Node.js ni Internet para abrir.
- **Rust** aparece sólo en la capa nativa de Tauri: gestiona el sidecar, la ventana y las funciones del kiosco como energía, batería y Wi-Fi. `src-tauri/src/kiosk.rs` incluye sus propias pruebas unitarias.
- **Python** es el idioma del sistema operativo: Debian lo trae, no requiere compilación y facilita herramientas de administración auditables.
- **Bash y PowerShell** automatizan la construcción de la ISO y el trabajo en Windows.
- **SQL** define el esquema del Worker en `migrations/`. El Worker aplica las migraciones solo en la primera solicitud.

5. Culinary Restaurant

Repositorio `MesaNexoNativo`. 16,398 líneas de producción y 6,701 de pruebas. La solución `MesaNexo.slnx` separa dominio, servicio, clientes y canales públicos.

5.1 Estructura

Proyecto	Tipo	Producción	Responsabilidad
`MesaNexo.Core`	Biblioteca net10.0	3,181	Reglas de negocio puras, modelo JSON, dinero, licencias, contratos de versión. Sin base de datos ni red.
`MesaNexo.Api`	Web net10.0	2,930	Backend de sucursal: HTTPS, SignalR, sesiones, almacén PostgreSQL, workers, licencias, integraciones reales.
`MesaNexo.Web`	React + Tauri	4,110	Cliente actual. Incluye `src-tauri` con `main.rs` y `kiosk.rs`.
`MesaNexo.Bridge`	Exe `culinary-bridge`	199	Sidecar: conserva sesión, bóveda y recibos fuera del WebView.
`MesaNexo.Transport`	Biblioteca	enlaza fuentes	Compila `Connection`, `LiveConnection` y `Updates` sin Avalonia.
`MesaNexo.Client`	Biblioteca Avalonia	1,695	Cliente nativo anterior y base móvil.
`MesaNexo.Desktop`	WinExe	67	Arranque Avalonia, bóveda DPAPI y Secret Service, descubrimiento Linux.
`MesaNexo.Android`	net10.0-android	71	API 31 o superior, Keystore y NSD. APK de depuración.
`MesaNexo.iOS`	net10.0-ios	49	Keychain y Bonjour. Sólo fuente, sin compilar.
`MesaNexo.Central`	Web net10.0	601	Receptor de eventos, multiempresa, espejos, analítica, pedidos en la nube.
`MesaNexo.Public`	Web net10.0	79	Portal público aparte: PWA, QR de mesa, factura y webhook de Deliverect.
`MesaNexo.Integrations`	Biblioteca	525	Adaptadores HTTP de proveedores. Ningún cliente la referencia.
`deploy/culinaryos`	Python y Shell	1,235	Instalador en el SO, kiosco, identidad de hardware, renovación TLS.
`tools`	PowerShell y Python	924	Laboratorio, compilación, empaquetado, evidencia, ISO.
`runtime`	Python	433	Extensión SQLite, exportador de migración y laboratorio.

5.2 Flujo de una mutación

Toda escritura del negocio sigue el mismo camino. Este flujo es el corazón de la confiabilidad del sistema.



Figura 5. Camino de una mutación, de izquierda a derecha por fila.

- **Repetición:** el mismo UUID con el mismo contenido devuelve el recibo original.
- **Abuso:** el mismo UUID con otro contenido se rechaza con `COS-DATA-IDEMPOTENCY`.

- **Respuesta perdida:** el cliente conserva el UUID, consulta `/api/operations/{id}` y no permite otra mutación hasta resolverlo. Esto también cubre la pérdida entre .NET y el WebView.
- **Conflicto:** el backend reintenta el CAS contra estado actualizado. El estado en memoria se reconstruye desde el almacén ante cualquier duda.
- **Lectura:** `/api/state` envía sólo lo que cambió desde lo que el cliente ya tiene. El historial se pagina.

5.3 Roles y permisos

Rol	Código	Acceso
Gerente	`manager`	Todas las áreas operativas, reportes, configuración, equipo, integraciones. 60 ramas de acción exigen este rol.
Mesero	`mesero`	Mesas, cuentas, comandas y pendientes. Permisos finos: cobrar, resultados, mostrador, unir mesas, transferir.
Cocina	`cocina`	Cocina en vivo e historial de preparación.
Cajero	`cajero`	Arqueo ciego.
Dueño	`dueno`	Analítica de la central, con origen y retraso de los datos.

Las contraseñas usan Argon2id. El PIN de autorización tiene 4 a 6 dígitos, se guarda ligado a sitio y usuario, y se bloquea cinco minutos tras cinco fallos. Reembolsos, cancelaciones y ajustes desde 1,000 MXN (configurable) requieren doble aprobación del gerente.

5.4 Funcionalidades

Área	Qué hace	Acciones y archivos
Mesas y cuentas	Zonas; estados libre, ocupada, impresa y sucia. Abrir, mover, unir, dividir por persona o N partes, transferir mesero, reabrir con motivo.	`visit.*`, `table.*`; `Restaurant.cs`, `Frontoffice.cs`, `PosExtensions.cs`
Comandas y modificadores	Grupos obligatorios y opcionales con costo, punto de cocción, notas saneadas, tiempos y etiquetas PARA LLEVAR o A DOMICILIO.	`command.send`, `unit.state`; `order-product-dialog.tsx`
Mostrador	Catálogo primero, carrito, comer aquí o llevar, atajos de efectivo y cambio. Cuenta, comanda y pago en un solo commit. Pantalla al cliente en segundo monitor.	`sale.quick`; `counter-page.tsx`, `customer-display.ts`
Cocina (KDS)	Unidades por estación, semáforos configurables y reloj, con estado recuperado del almacén.	`station.configure`, `Delivery.cs`
Caja	Apertura, movimientos, retiro parcial, cierre, límite de efectivo con bloqueo opcional, pulso de cajón sólo en efectivo.	`cash.*`, `drawer.request`
Arqueo ciego	El cajero cuenta sin ver el esperado. Las diferencias son de acceso restringido y avisan al dueño.	`BlindCash.cs`
Cortes y reportes	Corte X de lectura y Z con folio y digest. ABC por venta y cantidad. Exportación CSV.	`report.x`, `report.z`, `report.abc`; `Closings.cs`
Inventario	Múltiples almacenes, promedio ponderado o PEPS, capas, conteo físico, merma, traspasos y varianza entre costo teórico y real.	`stock.*`, `inventory.*`, `transfer.*`; `Inventory.cs`
Recetas	Recetas y subrecetas versionadas, rendimiento, conversión y detección de ciclos. Cada unidad congela su versión.	`recipe.*`
Compras	Proveedores con RFC, órdenes OC-0001, recepción contra orden, sugerencia mínimo-máximo.	`supplier.save`, `purchase.*`; `Backoffice.cs`
Cuentas por pagar y gastos	Vencimiento por días de crédito, pago parcial, antigüedad de saldos, cuentas de gasto con presupuesto, caja chica.	`payable.pay`, `expense.*`, `petty.*`
Clientes y lealtad	CRM, direcciones, cashback sobre lo pagado, monedero con recargas y pagos, listas de precios por canal y horario.	`customer.save`, `wallet.*`, `loyalty.configure`, `pricelist.save`; `Loyalty.cs`
Empleados	Consumo con descuento y tope mensual por nómina.	`staff.*`
Delivery	Pedidos, repartidor, polígonos y tarifas, hora programada de recoger, conciliación del repartidor.	`delivery.*`, `courier.save`, `Delivery.cs`
Pedidos y QR	Menú web, llamar al mesero y pedir por QR. Todo pedido público es una propuesta que el personal revisa.	`table.qr`, `qr.decide`; `TableQr.cs`, `CloudProposals.cs`
Fiscal	Solicitud, validación, timbrado CFDI 4.0, cancelación, conciliación y reintento con el mismo folio.	`pac.stamp`; `Fiscal.cs`, `FiscalProcessing.cs`
Reservaciones y evaluaciones	Bloque de dos horas; evaluación interna por visita.	`reservation.save`, `review.*`
Promociones	Combos y horarios en America/Mexico_City.	`promotion.save`
Soporte	Tickets contra Culinary Server y permiso temporal revocable.	`ticket.*`
Privacidad	Exportar y anonimizar un cliente conservando cifras.	`customer.export`, `customer.erase`; `Privacy.cs`

El dominio expone **122 acciones** agrupadas en 55 familias. El Anexo C las lista todas. La interfaz tiene 25 páginas: Resumen, Mesas, Mostrador, Cocina, Historial, Catálogo, Inventario, Compras, Gastos, Caja, Arqueo ciego, Reportes, Delivery, Clientes y monedero, Promociones, Reservaciones, Evaluaciones, Impresión, Fiscal, Integraciones, Analítica, Equipo, Configuración, Soporte y Pendientes.

5.5 Backend: rutas y workers

`MesaNexo.Api/ApiHost.cs` arma el servidor Kestrel: HTTPS, límite de 112 KiB por solicitud, 8 intentos de inicio de sesión por ventana y IP, cabeceras `no-store` y `nosniff`. El Anexo B lista las rutas.

Worker	Frecuencia o disparo	Qué hace
`InventoryWorker`	Trabajos en cola	Aplica consumos por ingrediente y capa con avance durable; resiste reinicio y dos procesos.

Worker	Frecuencia o disparo	Qué hace
`InventoryBalanceWorker`	Cada 6 horas	Concilia existencia contra movimientos, capas y reservas. Avisa; nunca corrige.
`OutboxWorker`	Periódico	Mantenimiento, archivo y alertas `COS-OUTBOX-SIZE` y `COS-OUTBOX-AGE`.
`CentralSyncWorker`	Con central configurada	Envía pendientes en orden FIFO con acuses por lote de hasta 16.
`CheckpointWorker`	Cada hora y al cerrar caja	Firma la cabeza de la bitácora y la publica en R2.
`OffsiteBackupWorker`	Cada 15 minutos si hubo cambios	Envía `pg_dump` con su SHA-256 a R2.
`LicenseWorker`	Al menos a diario	Renueva licencia, avisa vencimiento y aplica bloqueos firmados.
`CloudPull`	Conexión saliente mTLS	Recibe propuestas firmadas RSA-PSS y acusa decisiones ya confirmadas.
`PrintWorker`	Trabajos en cola	Consulta DLE EOT, imprime ESC/POS; una interrupción queda `uncertain`.
`FiscalWorker`	Solicitudes	Timbra en Facturama sin repetir folio.
`IntegrationWorker`	Uno por proveedor	Reclama con arrendamiento, envía una vez y consulta respuestas perdidas.
`CallerIdService`	Conexión AMI	Registra llamadas Asterisk con reconexión.

5.6 Almacén y cifrado

`PostgresBranchStore` guarda en cuatro tablas: `store_metadata`, `records`, `events` y `events_archive`. Las entidades no viven en tablas por tipo: cada una es un registro `mn_<tipo>_<id>` con versión, y `mn_meta_site` actúa como barrera CAS. Esto permite que PostgreSQL responda las mismas operaciones que el almacén SQLite de CulinaryOS (`transaction_v1`, `scan_v1`, `recent_v1`, `event_status_v1`, `outbox_status_v1`, `outbox_maintain_v1`).

Los datos se sellan con AES-256-GCM bajo el anillo de la sucursal, entregado por la credencial `keyring` de systemd. Una sucursal que venía de SQLite se migra una sola vez con `runtime/export_store.py`, mediante un pipe a `--migrate-from-export`. Los límites son deliberados: transacción de 56 KiB, 256 cambios y evento de hasta 64 KiB; una operación mayor se rechaza y debe dividirse.

5.7 Outbox y sincronización

- **Nunca se descarta un evento pendiente**, sin importar su edad ni si hay central. Una central conectada después recibe la historia completa.
- **Orden estricto.** El envío se detiene en el primer fallo. Un lote que dejaría atrás un evento anterior sin confirmar se rechaza completo.
- **Archivo.** Un evento con acuse de más de 7 días pasa a `events_archive`. `events + events_archive` siempre suma las transacciones confirmadas.
- **Alertas.** Tabla mayor de 2 GiB, o pendiente de más de un día con central configurada. La alerta avisa y nunca recorta.
- **Línea base.** `mesa-nexo-baseline-v1` existe sólo para sucursales cuya extensión anterior descartó pendientes sin central. Recupera el estado final, no la secuencia.

5.8 Bitácora y puntos de control

La auditoría encadena SHA-256 de cada cambio con su antes y después. Eso detecta una alteración aislada, pero quien tenga todas las llaves locales podría recalcular la cadena. Por eso `CheckpointWorker` publica la cabeza firmada en un bucket R2 de la sucursal con bucket lock, que rechaza borrar o sobrescribir. El nombre del objeto incluye secuencia y hash.

La sucursal compara al arrancar y a diario. Si no coincide, avisa al gerente, detiene las autorizaciones con PIN y deja evidencia en `alerts/`; la venta sigue. La revisión que cuenta es la externa: `--export-audit` en la sucursal y `--verify-checkpoints` en otro equipo, sin base de datos ni llaves. Cada restauración se declara con `--accept-checkpoints "motivo"`, y esa declaración queda visible en el bucket.

5.9 Licencias en la sucursal

- `Licensing.cs` verifica JWS Ed25519 sobre los bytes recibidos con la pública raíz incrustada en compilación (`-p:CulinaryRootPublicKey`).
- `LicenseKeys.cs` y `deploy/culinaryos/hardware-identity.py` ligan la licencia al equipo: llave no exportable en TPM 2.0, o huella de placa, disco y MAC.
- **Niveles:** Lite con 2 terminales fijas y Pro con 10 más reservaciones. Add-ons: `mobileOrdering`, `deliveryHub`, `webOrdering`. Los comanderos móviles no cuentan como terminales.
- **Gracia:** 7 días desde la última validación, firmados en `offlineUntil`. Nunca alarga una licencia vencida. Con licencia vencida o revocada sólo queda la pantalla de bloqueo, sin exportar datos.
- **Reloj:** se rechaza un reloj más de 15 minutos atrás de lo ya firmado. Por eso la hora de Internet usa NTS.

5.10 Integraciones

Cada integración tiene tres modos: apagada, simulación y real. El modo real sólo se puede elegir si existe un archivo válido en `/etc/culinaryos/integrations/<clave>.json` (`root:culinaryos-app`, `0640`). Sin archivo nada sale de la sucursal, y un archivo inválido marca al proveedor sin detener el backend.

Integración	Archivo	Proveedor real	Operación
Facturación	`pac.json`	Facturama (CFDI 4.0)	Timbra, consulta, cancela y envía PDF y XML
Terminal	`terminal.json`	Mercado Pago Point, Clip, BBVA semi-integrado	Cobra, consulta cada 3 s, cancela, devuelve. BBVA se registra desde el voucher
Pagos en línea	`onlinepay.json`	Stripe Checkout	Crea liga, consulta cada 10 s, aplica, reembolsa, expira
Mensajes	`messaging.json`	WhatsApp Cloud API, SMTP	Plantilla por evento, reintento sólo si no salió
Reparto	`aggregators.json`	Deliverect	Webhook HMAC en el portal, propuesta a sucursal, estados y menú
Hotel	`pms.json`	Cloudbeds, Oracle OPERA (OHIP)	Valida huésped y crédito, publica cargo
Mapas	`maps.json`	Google Geocoding y Routes	Ubica y mide distancia y tiempo
Llamadas	`callerid.json`	Asterisk AMI sobre TLS	Relaciona llamada y cliente por 10 dígitos
Impresión	`printer.json`	ESC/POS por red o USB	Cuentas, cortes, cajón RJ11
Bitácora y copias	`checkpoints.json`, `backups.json`	Cloudflare R2	Puntos de control y `pg_dump`

Regla común: el dominio deja el trabajo en cola y el servicio lo envía una vez con arrendamiento y llave de idempotencia. Una respuesta perdida no se reenvía: se consulta por referencia y, si no hay forma, queda «sin respuesta» para que una persona decida. El dinero sólo se mueve cuando el proveedor confirma. La API rechaza número de tarjeta, CVV y NIP. Las notificaciones FCM y APNs siguen aisladas y sin activar.

5.11 Cliente y kiosco

- **Bóveda.** DPAPI en Windows, Secret Service en Linux, Keystore en Android, Keychain en iOS. Sin alternativa en texto plano. La contraseña y el token nunca van a `localStorage`.
- **Puente.** El ejecutable lanza `culinary-bridge` con pipes privados. No hay servidor HTTP local. Los comandos de archivo exigen selector nativo; JavaScript no puede pasar rutas libres.
- **Kiosco.** Pantalla completa sin decoración y cierre bloqueado. Un supervisor reinicia la app si termina o si pasan 45 segundos sin señales, y registra la causa en `kiosk.log`.
- **Setup.** En una instalación nueva, un asistente crea restaurante y gerente, autoriza el equipo con un código de un uso y entra a operar.
- **Barra del kiosco.** Muestra red y Wi-Fi por `nmcli`, y batería o UPS sólo cuando el equipo opera sin corriente. Incluye botón de energía.

- **Otras plataformas.** Android compila un APK de depuración sin haberse ejecutado en dispositivo. iOS es sólo fuente.

5.12 Central y portal público

`MesaNexo.Central` recibe eventos por mTLS y responde ACK después del commit durable, con deduplicación por sucursal y evento. Sus tablas son `event`, `tenant`, `tenant_branch`, `tenant_user`, `membership`, `tenant_session`, `mirror`, `cloud_order`, `cloud_command`, `webhook_receipt`, `invoice_capability` y `public_attempt`, con llaves compuestas y cifrado ligado a empresa, sucursal, tipo e identificador. Tiene su propia clave AES de 32 bytes, distinta de las sucursales.

`MesaNexo.Public` es un proceso aparte que sirve la PWA por `/s/{slug}`, el catálogo, pedidos, QR de mesa, factura y `/webhooks/deliverect/{slug}`. No contiene rutas privadas de sucursal. El precio se calcula en el servidor y el resultado es una propuesta: un pedido público no es venta hasta que el gerente lo acepta y se revalidan precio, cupón y existencias en la misma transacción.

5.13 Reglas de dinero

`src/MesaNexo.Core/Cents.cs` es el único lugar que redondea importes de venta. Las reglas están en `docs/POLITICA-MONETARIA.md` y requieren revisión de un contador antes de facturar en real.

Tema	Regla
Unidad	Centavos enteros de MXN en `long`. Nunca flotantes.
Precios	Incluyen IVA. La tasa se congela en cada unidad vendida.
IVA incluido	`importe × tasa / (10000 + tasa)`, por unidad, sobre su parte neta de descuento.
Reparto	Cada parte recibe su piso y los centavos restantes van a los residuos mayores. Siempre suma el total.
Propina	Va aparte: fuera de venta, IVA y CFDI; dentro de la caja de su forma de pago.
Devoluciones	Registro inmutable que apunta al pago. Efectivo sale de caja; terminal o liga pasa a `refund-queued`; sin API queda `refund-manual`.
Inventario	Sin existencias negativas. Se reserva al enviar a cocina y se consume al liquidar con la receta congelada.

6. CulinaryOS

Repositorio `CulinaryOS`. 3,639 líneas de producción y 1,420 de pruebas. Es Debian 13 trixie amd64 sin modificar el kernel, más la capa propia de servicios, seguridad y operación. La imagen integrada 0.3.0-dev trae Culinary Restaurant dentro de la ISO.

6.1 Estructura

Ruta	Contenido
`config/`	`project.json`, `appliance.json`, listas de paquetes (`base`, `terminal`, `servidor`, `sucursal`, `appliance`), preseed, gestores de arranque, plan de red.
`os/overlay/`	Archivos que se copian al sistema: `usr/local/lib/culinaryos`, comandos en `usr/local/sbin`, unidades systemd, marca.
`os/appliance/`	Lo exclusivo de la imagen integrada: `appliance.py`, `recovery.py`, comandos de recuperación.
`os/hooks/`	Ganchos de chroot de live-build: `0100-culinaryos`, `0200-appliance`.
`scripts/`	`build.sh`, `build-appliance.sh`, `audit-appliance.py`, `prepare-appliance-input.py`, `install-lab.sh`, `run-vm.sh`, reparaciones de laboratorio, `verify.ps1`.
`tests/`	11 suites de `unittest` con 185 pruebas.
`docs/`, `results/`	Contratos, operación y resultados de validación.

6.2 Módulos

Módulo	Líneas	Función
`apphost.py`	109	Lanza backend y UI registrados, sin shell, y comunica readiness
`backup.py`	137	Copia de configuración con lista positiva
`bootconfig.py`	52	Opciones de arranque idempotentes
`branding.py`	48	Identidad visual del sistema
`dataio.py`	117	Cliente y servidor IPC del almacén
`deployment.py`	112	Despliegue de red de sitio con reversión
`discovery.py`	46	Verifica candidatos mDNS contra TLS
`health.py`	221	Diagnóstico público, sin secretos
`lan.py`	139	Indicadores `COS-LAN-*`
`pki.py`	156	CA local y certificados por OpenSSL
`provision.py`	208	Aprovisiona sucursal, claves y `culinaryos-runtime`
`runtimehealth.py`	57	Indicadores del runtime
`securedata.py`	105	XChaCha20-Poly1305, Argon2id, anillo de claves
`site.py`	160	Valida el plan y genera perfiles de red
`store.py`	158	Almacén SQLite cifrado y outbox
`syncworker.py`	44	Emisor original hacia la central
`ups.py`	92	Prepara e instala NUT
`welcome.py`	59	Pantalla de bienvenida
`appliance.py`	143	Primer inicio de la imagen integrada
`recovery.py`	177	Kit de recuperación y restauración

6.3 Perfiles

Perfil	Contenido	Uso
`terminal`	Escritorio, sin base local	Puesto de meseros o cocina
`servidor`	Base y servicios, sin escritorio	Servidor dedicado
`sucursal`	Escritorio y servicios en el mismo Debian	Equipo único
`appliance`	Sucursal más aplicación, PostgreSQL, kiosco	ISO instalable 0.3.0-dev

6.4 Instalación y primer inicio



Figura 6. De la ISO a la operación. Cada instalación genera su propia identidad; la imagen no trae cuentas ni datos.

El constructor pide la contraseña inicial de la cuenta administradora `devmode` y guarda sólo su hash SHA-512 en el preseed. El host queda como `RunTimeOSDev`. El usuario `operador` queda reservado para la aplicación. El modo inicial usa loopback con HTTPS verificado y no abre el puerto de la aplicación en la LAN.

6.5 Servicios

Unidad	Función
`culinaryos-firstboot`	Alta local, cuentas propias; LightDM depende de que termine
`culinaryos-data`	Almacén SQLite por socket Unix. En la imagen integrada sólo responde `health`
`culinaryos-backend`	Lanza el backend registrado. `Type=notify`, sólo tras readiness real
`culinaryos-publish`	Anuncia DNS-SD únicamente si hay backend disponible
`culinaryos-snapshot`	`pg_dump` con su SHA-256 en `/var/lib/culinaryos-app/snapshots`
`culinaryos-renew-tls.timer`	Diario a las 04:00; renueva a menos de 35 días del vencimiento
`culinaryos-health.timer`	Diagnóstico acotado, desde 45 s tras arrancar
`culinaryos-sync.timer`	Emisor del perfil genérico. Enmascarado en la imagen integrada
`culinaryos-maintenance`	Aviso de mantenimiento sin abrir consola root
`culinaryos-rekey`	Recifrado administrativo del almacén

Aislamiento del backend, de `culinaryos-backend.service`:

```
User=culinaryos-app        ProtectSystem=strict     ProtectHome=yes       PrivateTmp=yes
PrivateDevices=yes         NoNewPrivileges=yes      RestrictSUIDSGID=yes  CapabilityBoundingSet=
RestrictAddressFamilies=AF_UNIX AF_INET AF_INET6 AF_NETLINK
MemoryMax=768M             TasksMax=128             CPUQuota=150%         LimitCORE=0
```

6.6 Red

- Referencia `10.77.0.0/24`: servidor en `.10`, meseros en `.101–.119`, cocina en `.121–.139`, gerente en `.141–.159`. Nombre interno `culinaryos.internal`.
- `nftables`: entrada y reenvío en `drop`, sin SSH. El registro de la aplicación abre sólo su puerto desde la interfaz operativa.
- Avahi por interfaz publica `_culinaryos._tcp` sólo cuando hay backend real. Un anuncio nunca concede confianza.

- `chrony` toma hora autenticada por NTS (`time.cloudflare.com`, `nts.netnod.se`) y la sirve a la LAN con estrato local 10.
- `culinaryos-firewall-apply` y `nmcli device checkpoint` revierten un cambio de red no confirmado.
- Diagnóstico separado por capa: enlace, dirección, servidor, nombre, servicio, TLS, puerta de enlace e Internet (`COS-LAN-*`).

6.7 Seguridad y cifrado

Material	Dónde	Protección
Llavero de datos	`/etc/culinaryos/secrets/branch.keys.json`	root 0600; se entrega por `LoadCredential`. Rotación con `culinaryos-runtime rotate`
Almacén SQLite heredado	`/var/lib/culinaryos-data/store.sqlite`	XChaCha20-Poly1305 IETF, AAD con sitio, propósito y versión. WAL y `synchronous=FULL`
CA del restaurante	`restaurant-ca.crt`	10 años; rotación manual en el año 9, con aviso desde 365 días antes
Certificado TLS	`restaurant-tls.crt`	365 días, renovación automática atómica; la app avisa desde 30 días antes
Copia del anillo	Externa	Argon2id INTERACTIVE, salt de 16 bytes, mismo AEAD
Kit de recuperación	Externo	Contraseña de 16 a 1024 caracteres, guardada aparte
Identidad de licencia	TPM o huella	No se copia; se reaprovisiona en otro equipo

Pendiente de sistema: LUKS y Secure Boot no están validados en una instalación final. El cifrado por registro protege un volcado, pero quien tenga disco y anillo puede descifrar.

6.8 Respaldo y recuperación

Objetivos: **RTO de 4 horas y RPO de 15 minutos**, todavía sin medir. `culinaryos-recovery-kit` guarda llavero, identidad, CA, certificados y configuración de integraciones. La base va por `pg_dump` y por la copia a R2 cada 15 minutos. Licencia, identidad de hardware y cuentas de Linux se reaprovisionan. `--verify-store` comprueba la base restaurada y el proceso declara la restauración en los puntos de control (sección 9.5).

6.9 UPS y operación física

`culinaryos-ups` prepara NUT en dos fases: primero comunicación y después monitor de apagado. `upsd` escucha sólo loopback y usa un secreto aleatorio local. El diagnóstico incluye SMART y temperatura. No se ha probado un UPS físico.

7. Culinary Server

Repositorio `CulinaryServer`. 1,660 líneas de producción y 756 de pruebas. Cloudflare Worker en TypeScript con Hono. Sólo atiende a dos clientes: cualquier otra ruta responde 404, y CORS sólo acepta los orígenes de `ADMIN_ORIGINS`.

7.1 Estructura

Archivo	Líneas	Función
`src/index.ts`	37	Monta rutas, CORS, errores, cron
`src/admin.ts`	209	Licencias, versiones, publicaciones, tickets, registro
`src/admin-auth.ts`	160	Alta, sesión, TOTP, invitaciones, cuentas
`src/branch.ts`	113	Rutas de sucursal
`src/updates.ts`	195	Subida a R2, manifiestos, autorización de actualización
`src/crypto.ts`	138	Ed25519, ES256, SHA, PBKDF2
`src/keys.ts`	90	Intermedias, cadena, piso de llaves
`src/licenses.ts`	134	Emisión de licencias y avisos
`src/signature.ts`	54	Retos y firma de solicitudes
`src/tickets.ts`	77	Soporte
`src/db.ts`	44	Migraciones y purga
`src/http.ts`	77	Errores y utilidades
`src/catalog.ts`	39	Planes, niveles, add-ons, funciones

7.2 Modelo de datos

Tabla D1	Contenido
`license`	Cliente, sitio, plan, nivel, add-ons, límites, revisión, llave, huella, certificado mTLS, equipo liberado, baja lógica
`admin`, `admin_invite`, `admin_session`	Cuentas por dispositivo, hash de contraseña, TOTP, bloqueo, invitaciones, sesiones por hash
`release`, `publication`, `publication_target`	Versiones por plataforma y su destino
`download_grant`	Permisos de descarga de una hora
`ticket`, `ticket_message`	Soporte
`used_challenge`	Retos consumidos
`audit`	Bitácora del panel y de firmas
`d1_migrations`	Control de migraciones

Un cron diario (`17 4 * * *`) purga lo vencido. Los paquetes viven en el bucket R2 `culinary-server-packages`. Requiere plan Workers Paid, porque el SHA-512 del paquete y PBKDF2 superan el límite de CPU del plan Free.

7.3 Cadena de llaves



Figura 7. Quién firma qué. La raíz además firma el piso de llaves (`culinary-key-floor+jws`), que revoca generaciones comprometidas. La sucursal exige que autorización y manifiesto coincidan en versión, archivo, tamaño y SHA-512.

Llave	Vive en	Si se compromete
Raíz	Gestor de contraseñas o USB cifrada	Raíz nueva y reaprovisionar todas las sucursales
Licencias	`LICENSE_SIGNING_KEY`, `LICENSE_SIGNING_CHAIN`	Subir piso con `sign-key-floor.mjs` y firmar otra generación
Paquetes	Estación de firma	Igual que licencias
Retos	`CHALLENGE_KEY`	Generar otra; sólo invalida retos en curso

7.4 Solicitudes firmadas

Cada solicitud firmada usa un reto de 2 minutos y un solo uso, emitido sin guardar estado en el servidor. Se firma esta cadena con la llave de la licencia o del dispositivo:

```
culinary-request-v1
<scope>             branch | admin
<MÉTODO>            GET, POST, PUT
<ruta>              /branch/v1/license/renew
<subject>           licenseId o username
<nonce>             nonce del reto
<sha256(cuerpo)>    hexadecimal de los bytes exactos
```

7.5 Reglas de licencia

- Planes `trial`, `monthly`, `annual`, `permanent`. Estados `pending`, `active`, `expired`, `revoked`. Revocar es definitivo.
- Sin TPM deben coincidir al menos dos de tres partes de la huella. Cada cambio aceptado queda en auditoría.
- Transferir (`reset-activation`) exige motivo. El último equipo liberado recibe un aviso firmado y se bloquea en su siguiente contacto.
- Un aviso de bloqueo sólo vale con firma válida, el `licenseId` y `siteId` propios, el nonce de esa solicitud y revisión no menor. Un 403 sin aviso válido no bloquea.
- Límites de soporte: 20 tickets abiertos por licencia y 100 mensajes por ticket.

7.6 Herramientas

Script	Uso
`generate-secrets.mjs`	Genera raíz, reto y código de alta sin sobrescribir
`sign-intermediate.mjs`	Firma una intermedia `license` o `release`
`sign-key-floor.mjs`	Firma el piso de llaves
`sign-release.mjs`	Firma el manifiesto de un paquete en la estación

8. Culinary Admin

Repositorio `CulinaryAdmin`, versión 0.1.3. 950 líneas de producción y 241 de pruebas. Panel React empaquetado con Tauri para Windows (NSIS) y Android (APK ARM64). La capa nativa sólo arranca la ventana: el WebView llama directo a Culinary Server y nada nativo queda expuesto. Sus 9 escenarios de Playwright corren en Edge contra un servidor simulado.

Archivo	Líneas	Función
`src/api.ts`	137	Cliente `/admin/v1`, llave del dispositivo y firma
`src/gate.tsx`	80	Alta, confirmación e inicio de sesión
`src/licenses.tsx`	153	Tablero de licencias y sus acciones
`src/releases.tsx`	66	Subida por partes, firma y publicación
`src/support.tsx`	28	Tickets
`src/team.tsx`	39	Invitaciones, cuentas, llave raíz y bitácora
`src/ui.tsx`	52	Componentes
`src/App.tsx`	40	Navegación
`src/theme.css`	229	Sistema visual

8.1 Funcionalidades

- **Acceso de tres factores.** Llave de dispositivo no exportable (WebCrypto), contraseña y TOTP. Cinco fallos bloquean 15 minutos y la sesión dura 8 horas.
- **Licencias.** Crear, editar, renovar, revocar, liberar equipo, cambiar `siteId` sin equipo ligado y eliminar. Combos como atajo. El código de activación se muestra una vez.
- **Versiones.** Subida multiparte a R2, firma pegada desde la estación, publicación a todas o a licencias elegidas, y retiro.
- **Soporte.** Responder y cerrar tickets de las sucursales.
- **Equipo y seguridad.** Invitaciones, cuentas, llave pública raíz, vencimiento de la intermedia y bitácora. Crear invitaciones y borrar cuentas pide una firma fresca del dispositivo.

8.2 Seguridad y diseño

La política de contenido restringe scripts a `self` y conexiones a `https:`. Las acciones irreversibles nombran la licencia y piden escribir su `siteId`. Los secretos de un uso se muestran una vez. El diseño sigue una metáfora de tablero de llaves de hotel: negro lacado y rojo para acciones, nunca para un estado normal; Manrope para interfaz y JetBrains Mono para datos.

8.3 Compilación

- `tools/build-windows.ps1` genera el instalador NSIS reutilizando Rust y MSVC de `MesaNexoNativo/.tools`.
- `tools/build-android.ps1 -Release` genera un APK ARM64 con JDK 17, SDK 34 y NDK 27. Se firma con una llave local de prueba: una tienda necesita llave propia.

9. Workflows de punta a punta

9.1 Vender en el comedor



Figura 8. Ciclo de una cuenta. Una comanda sin insumos suficientes se rechaza antes de escribir nada.

9.2 Cobro con terminal y resultado incierto

1. `terminal.charge` crea el intento y bloquea otro cobro sobre esa cuenta.
2. `IntegrationWorker` lo reclama con arrendamiento y lo envía con llave de idempotencia.
3. El servicio consulta el estado cada 3 segundos.
4. **Aprobado:** se aplica el pago. **Rechazado:** no hay efecto.
5. **Sin respuesta:** queda incierto. `terminal.reconcile` aplica el cargo una sola vez, o lo revierte si la cuenta ya se cubrió por otro medio.

9.3 Activar y renovar una licencia



Figura 9. Activación y renovación. Si la activación se rechaza, el código no se gasta.

9.4 Publicar una actualización

1. En Culinary Admin se crea la versión y se sube el paquete por partes a R2.
2. Al completar, el Worker calcula el SHA-512 de lo que quedó en R2 y deja la versión en `signing`.
3. En la estación: `node scripts/sign-release.mjs --file … --release-id … --version … --platform ….`
4. Se pega la firma en el panel. El Worker verifica cadena y bytes, y la versión pasa a `ready`.
5. `POST /admin/v1/publications` la dirige a todas o a licencias concretas.
6. La sucursal consulta, recibe autorización y manifiesto, y descarga con un permiso de una hora, reanudable.
7. Antes de instalar se verifican ambas firmas, versión mayor, tamaño y SHA-512. Requiere cuentas y sesiones cerradas.

8. Si cambia el esquema: `--upgrade-schema` crea una copia con `pg_dump`, verifica que sea de esa sucursal y migra. Revertir significa volver a la copia.

9.5 Sustituir un servidor dañado

1. Instalar la misma versión de la ISO o una más nueva y completar el primer arranque, sin crear restaurante en Setup. `culinaryos-restore` rechaza un equipo que ya tenga uno.
2. Descargar de R2 la copia más reciente con su `.sha256`, o usar la que se sacó a mano.
3. `sudo culinaryos-restore --kit … --dump …`: abre el kit, compara el SHA-256, carga la copia en la base temporal `culinaryos_restore` y ejecuta `--verify-store` con el llavero del kit.
4. En Culinary Admin, **Liberar equipo** con motivo. Si el equipo anterior sigue encendido, se bloquea en su siguiente contacto.
5. Activar la licencia en el equipo nuevo con el código nuevo. Hasta entonces la app muestra la pantalla de activación.
6. Si el sitio no es `local_only`, reaplicar red y firewall del plan de sitio.
7. Declarar en los puntos de control que hubo restauración (`--accept-checkpoints`).
8. Comprobar ventas, caja e inventario, hacer una venta de prueba y crear un kit nuevo si cambió algo.
9. Las terminales siguen confiando porque la CA no cambió. Lo vendido después de la copia se concilia a mano.

9.6 Sin Internet o sin central

Falla	Comportamiento
Internet	La venta sigue. Pendientes de outbox se conservan. Timbre y cobros en línea quedan en cola.
Central caída	La sucursal sigue. Acuses pendientes; alerta a más de un día.
Servicio de licencias	Gracia de 7 días desde la última validación; aviso antes de vencer.
Terminal LAN	Las demás siguen. Solicitud pendiente con su UUID hasta reconciliar.
Servidor de sucursal	La operación se detiene. No hay modo autónomo en terminales. UPS y procedimiento 9.5 son la respuesta.
Impresora sin papel	El trabajo queda pendiente; una impresión interrumpida queda incierta y nunca se repite sola.
Corriente	PostgreSQL local y UPS si está configurado. El corte eléctrico real sigue sin probarse.

9.7 Construir la ISO

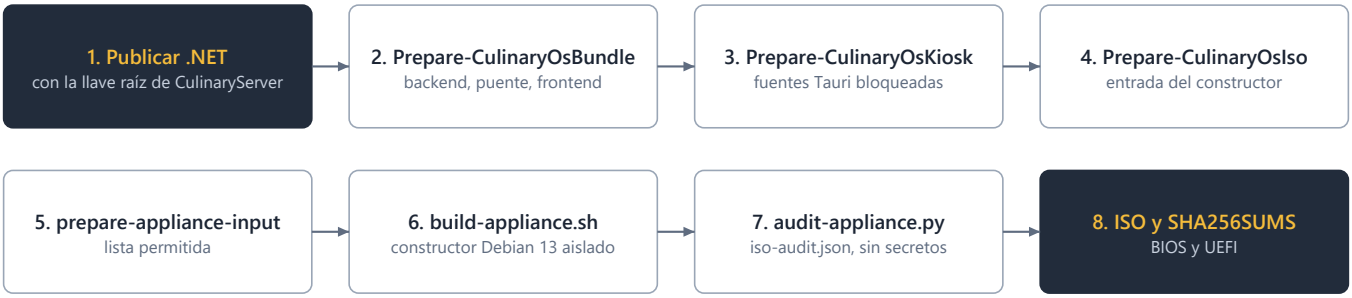


Figura 10. Cadena entre repositorios para producir la imagen integrada.

9.8 Instalar la aplicación en un CulinaryOS existente

`deploy/culinaryos/install.py` ejecuta, en orden lógico: `preflight` y `validate_runtime` (hashes del runtime y extensiones conocidas), `copy_files`, `certificates`, `postgres_database`, `migrate_sqlite`, `initialize_branch`, `backend_dropin`, `single_outbox_sender`, `tls_renewal` e `integrations_directory`. Respalda antes de cambiar y rechaza otra aplicación registrada.

10. Ramas, una por una

Cada fila usa el contenido exacto de esa rama. Varias ramas comparten árbol: la misma foto de archivos con distinto historial. Se indica cuando sucede. «Producción» y «Pruebas» son líneas de código con el criterio de la sección 3.

10.1 Culinary Restaurant

Rama	Commit	Fecha	Commits	Archivos	Producción	Pruebas	Docs	Bytes prod.
estado-en-memoria	1b278a5	10-23	8	477	9,569	2,429	1,031	1,234,897
main	ef302b6	10-23	9	477	9,569	2,429	1,031	1,234,897
impresora-escpos	ca939fc	10-23	10	481	9,753	2,550	1,051	1,251,545
facturacion-sandbox	72b38de	10-23	12	485	9,816	2,710	1,069	1,259,641
integraciones-produccion	6c7c937	10-25	30	521	14,194	4,821	1,104	1,506,214
hardware-ui-pila-atajos	7be48dc	10-30	49	563	15,914	5,644	1,411	1,659,097
claude/hardware-ui-pila-atajos-3c01b6	425d1e3	10-30	49	555	16,041	5,724	1,425	1,665,999
topbar-parpadeo-pila	03378af	10-30	58	579	16,356	6,654	1,717	1,696,326
origin/postgresql	b838bba	10-31	59	579	16,356	6,654	1,717	1,696,326
origin/main	810e518	10-31	66	579	16,356	6,654	1,717	1,696,326
postgresql	d8fc1ca	10-31	60	591	16,398	6,701	1,719	1,699,398

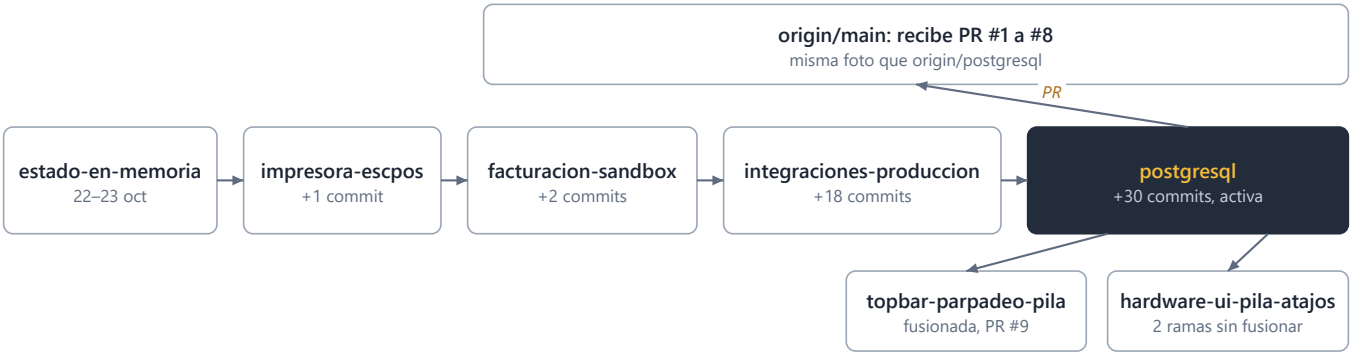


Figura 11. Las ramas se apilan: cada etapa parte de la anterior y se fusiona a main por PR.

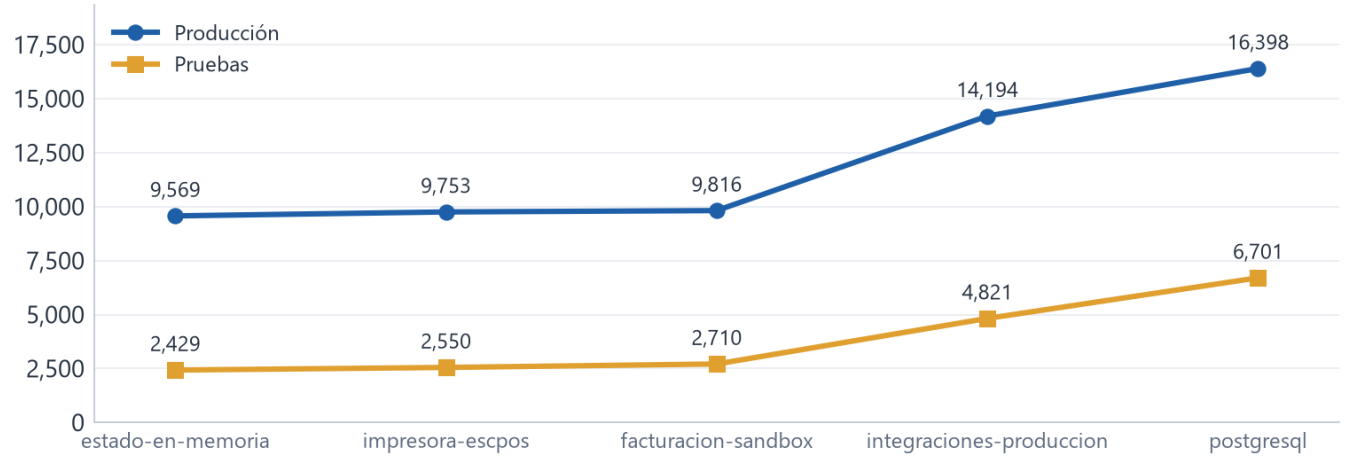


Figura 12. Crecimiento acumulado por etapa. El salto de integraciones-produccion incluye el commit a0c653d, que uniformó espaciado en C#, TS y Python: en bytes de producción el aumento es de 20 %, no de 45 %.

Rama	Alcance	Qué aporta	Estado
`estado-en-memoria`	Rama predeterminada del remoto (`origin/HEAD`) y base histórica.	Estado confirmado en memoria con un solo escritor; `/api/state` por deltas; estado incremental; outbox acotado; arranque rápido del almacén. Fue la mejora de rendimiento clave.	Fusionada por PR #1. `main` local es sólo ese merge.
`main` (local)	Merge del PR #1 sobre `estado-en-memoria`.	Mismo árbol que `estado-en-memoria`.	Atrasada: `origin/main` lleva 57 commits más.
`origin/main`	Línea de integración publicada (`810e518`).	Recibe los PR #1 a #8. Misma foto que `origin/postgresql` y `topbar-parpadeo-pila`.	Falta `d8fc1ca`.
`impresora-escpos`	+1 commit; 11 archivos, +502 / −7.	Impresora ESC/POS, servicio de impresión, DLE EOT y cajón RJ11.	Fusionada por PR #2.
`facturacion-sandbox`	+2 commits; 26 archivos, +590 / −100.	CFDI 4.0 con sandbox de Facturama como servicio; botón «Reintentar».	Fusionada por PR #3.
`integraciones-produccion`	+18 commits; 233 archivos, +17,561 / −8,217.	Modo «Real» de todas las integraciones; licencias con Culinary Server; salida de funciones de proveedor; Lite, Pro y add-ons; formato uniforme.	Fusionada por PR #4, #5 y #6.
`postgresql`	+30 commits; 275 archivos, +8,283 / −1,327. **Rama de trabajo.**	PostgreSQL en sucursal y central; envío de outbox desde el backend; kiosco con energía y red; auditoría 0.3.0: cadena de llaves, pagos inciertos, gracia, TLS, puntos de control, copia externa, política monetaria, doble aprobación, privacidad y CI por riesgo.	PR #7 y #8 fusionados. `d8fc1ca` sigue sin subir.
`topbar-parpadeo-pila`	Sin commits propios frente a `postgresql`.	Barra superior sin encimarse, cambio de pantalla sin parpadeo e indicador de pila.	Fusionada por PR #9. Se puede archivar.
`hardware-ui-pila-atajos`	+1 commit, 12 atrás; 18 archivos, +239 / −18.	Intento local del mismo cambio de barra, con el mismo mensaje.	Sin subir. Reemplazada por `topbar-parpadeo-pila`.
`claude/hardware-ui-pila-atajos-3c01b6`	+1 commit, 12 atrás; 12 archivos, +516 / −31.	Variante del mismo cambio creada en un worktree de sesión.	Sin subir. Reemplazada igual.

Lenguajes por rama (producción y pruebas, líneas de código)

Rama	C#	Python	TS	TSX	CSS	PS	Rust	Shell	Total
`estado-en-memoria`	5,260	2,309	563	1,893	1,136	416	141	61	11,998
`main`	5,260	2,309	563	1,893	1,136	416	141	61	11,998
`impresora-escpos`	5,565	2,309	563	1,893	1,136	416	141	61	12,303
`facturacion-sandbox`	5,764	2,309	586	1,894	1,136	416	141	61	12,526
`integraciones-produccion`	11,015	2,895	1,016	2,068	1,138	441	130	62	19,015
`hardware-ui-pila-atajos`	12,662	3,157	1,187	2,244	1,175	466	355	62	21,558
`claude/hardware-ui-pila-atajos-3c01b6`	12,662	3,248	1,254	2,251	1,174	466	398	62	21,765
`topbar-parpadeo-pila`	13,299	3,771	1,266	2,265	1,174	466	398	62	23,010
`origin/postgresql`	13,299	3,771	1,266	2,265	1,174	466	398	62	23,010
`origin/main`	13,299	3,771	1,266	2,265	1,174	466	398	62	23,010
`postgresql`	13,347	3,812	1,266	2,265	1,174	466	398	62	23,099

10.2 CulinaryOS

Rama	Commit	Fecha	Commits	Archivos	Producción	Pruebas	Docs	Qué es
`main` (local)	3c1af1d	10-24	1	168	3,361	1,203	1,621	Versión inicial. Atrasada frente a `origin/main`.
`postgresql`	c753621	10-30	10	176	3,639	1,420	1,750	9 commits; 31 archivos, +747 / −43. PostgreSQL, polkit, Wi-Fi, NTS, restauración, SMART.
`origin/main`	dde61e6	10-31	12	176	3,639	1,420	1,750	Merge del PR #2. Mismo contenido que `postgresql`.

10.3 Culinary Admin y Culinary Server

Repositorio y rama	Commit	Fecha	Commits	Archivos	Producción	Pruebas	Docs	Nota
Admin `master`	329bbcd	10-30	12	87	950	241	141	Rama única, sincronizada con el remoto.
Server `main`	d627abf	10-30	18	41	1,660	756	666	Rama única. Cada push despliega a producción en Cloudflare.

Ambos repositorios trabajan directo sobre su rama principal, sin PR.

11. Metodologías de trabajo

No hay un marco formal como Scrum o Kanban en los repositorios: no aparecen sprints, tablero ni estimaciones. La forma de trabajar se deduce del historial y de los documentos, y es consistente.

Práctica	Cómo se aplica	Evidencia
Entrega por etapas	Cuatro etapas sobre la hoja «Diagrama maestro»: punto de venta, administración, clientes y cortes, integraciones simuladas. Cada una cierra con pruebas y capturas.	`docs/DEVMap-REQUISITOS.md`
Trazabilidad de requisitos	Matriz con ID, estado real, archivos y pruebas por requisito. «Parcial» se distingue de «Implementado».	`DEVMap-REQUISITOS.md`, `PARIDAD.md`
Auditoría por hallazgos	18 hallazgos contrastados con el código, en tres fases, cada uno con acción y comprobación.	`docs/PLAN-AUDITORIA-0.3.0.md`
Madurez en cuatro estados	Implementado, Probado, Validado en real, Habilitado. Sólo el último se vende.	`docs/ESTADO-COMERCIAL.md`
Ramas apiladas con PR	Una rama por capacidad, fusionada a main por PR. En Culinary Restaurant y CulinaryOS.	PR #1 a #9
Pruebas guiadas por riesgo	Evidencia agrupada por Dinero, Inventario, Seguridad, Restauración y Sincronización.	`tools/evidence_report.py`
Evidencia reproducible	Cada suite escribe JSON. CI borra la evidencia anterior y sube el resultado como artefacto.	`evidence/`, `.github/workflows/ci.yml`
Simulador primero	Cada integración recorre los mismos registros simulada y real. La prueba real se deja al final, por decisión expresa.	`Core/Integrations.cs`, `PROVEEDORES.md`
Pruebas de falla	Se inyectan pérdida de respuesta, disco lleno, permisos, doble worker y caídas.	`BoundaryTests.cs`, `FaultStore`
Contratos versionados	API, esquema, eventos y extensión tienen número y regla de cambio.	`Contract.cs`, `COMPATIBILIDAD.md`
Medición de carga	Línea base, cambio y nueva medición con el mismo escenario.	`evidence/load-*.json`
Registro de decisiones	Decisiones del usuario con fecha y motivo dentro de los documentos.	`INTEGRACIONES-REVISION.md`
Diseño aprobado antes de integrar	Maqueta, revisión y traslado a componentes, con reglas escritas.	`DISENO-APROBADO.md`, `DESIGN.md`
Desarrollo asistido por IA	Agentes con habilidades de diseño y revisión de código, más prompts de integración en la documentación.	`.claude/`, `.impeccable/`, `PROMPT-ECOSISTEMA-…`

Patrones de arquitectura

Patrón	Dónde
Outbox transaccional	`PostgresStore.cs`, `CentralSyncWorker.cs`
Idempotencia por UUID y huella	`BranchService`, recibos, `event_id`
Concurrencia optimista (CAS)	Versión por registro, barrera `mn_meta_site`
Escritor único	`BranchService` con semáforo
Puertos y adaptadores	`IPacAdapter`, `IPrintDriver`, `MesaNexo.Integrations`
Sidecar	`culinary-bridge` junto a Tauri
Máquina de estados de pagos	`PaymentStates.cs`
Reclamo con arrendamiento	`IntegrationWorker`, trabajos durables
Cadena de hash y anclaje externo	`AuditChain.cs`, `Checkpoints.cs`
Delta de estado	`StateDelta.cs`
Propuesta y aceptación local	`CloudProposals.cs`
Mínimo privilegio	Usuarios de servicio, `LoadCredential`, aislamiento systemd

Convenciones

- Mensajes de commit en español que describen el efecto.
- Versiones de dependencias fijadas y archivos de bloqueo en NuGet y npm.
- Sin cuentas de demostración, llaves ni credenciales predeterminadas en los productos.
- Límites explícitos: una operación demasiado grande se rechaza, no se trunca.

12. Pruebas y CI

Última corrida registrada de la suite de integración: **976 de 976 comprobaciones aprobadas**, el 31 de octubre de 2026, según `evidence/integration.json`. Esa corrida y las 28 pruebas del runtime se repitieron después de corregir las discrepancias de la sección 13, sin fallos.

Suite	Tipo	Cantidad	Dónde
Integración .NET	API HTTPS real, PostgreSQL privado, procesos y UI headless	976 comprobaciones	`tests/MesaNexo.Tests`
Runtime	Extensión Python, vectores, fallas inyectadas	28 pruebas	`tests/test_runtime_extension.py`
Instalador	Python con sistema simulado	48 pruebas	`tests/test_culinaryos_install.py`
Kiosco	Python	17 pruebas	`tests/test_culinaryos_kiosk.py`
Puente	HTTPS, runtime y DPAPI reales	24 comprobaciones	`tests/MesaNexo.Hybrid.Tests`
Web	Playwright contra backend real	37 escenarios	`src/MesaNexo.Web/tests`
Kiosco nativo	Rust, batería y UPS	pruebas en `kiosk.rs`	`src-tauri/src/kiosk.rs`
CulinaryOS	`unittest`	185 pruebas	`CulinaryOS/tests`
Culinary Server	Vitest en runtime de Workers	64 pruebas	`CulinaryServer/test`
Culinary Admin	Playwright con servidor simulado	9 escenarios	`CulinaryAdmin/tests`

Cantidades de runtime y puente tomadas de la evidencia. Instalador, kiosco, Web, CulinaryOS, Server y Admin se contaron en el código.

Cobertura por riesgo

Suite	Comprobaciones	Riesgo
`money`	145	Dinero: cuentas aleatorias al centavo
`licenses`	92	Seguridad
`live-integrations`	62	Dinero, contra APIs simuladas
`backoffice`	48	Inventario y compras
`store-outbox`	42	Restauración y sincronización
`checkpoints`	38	Seguridad y restauración
`approvals`	32	Dinero y seguridad
`fiscal-sandbox`	31	Dinero, Facturama simulado
`tenants`	27	Aislamiento multiempresa
`inventory`	17	Inventario
`privacy`	11	Privacidad
`boundaries`	10	Fallas en cada frontera

Las suites temáticas son parte de las 976. No deben sumarse como pruebas adicionales.

Integración continua

`.github/workflows/ci.yml` existe sólo en Culinary Restaurant. Corre en `windows-latest` con un tope de 60 minutos, en push a `postgresql` y `estado-en-memoria`, y en todo PR.



Figura 13. Pasos de CI.

Carga

Escenario `tests/MesaNexo.Load`: 6 meseros, 2 de cocina, 1 gerente, 24 mesas, sondeo cada 2 segundos y 15 minutos sobre el backend real.

Medición	Fecha	Resultado	Registros	Cuentas	p50 estado	p95 estado
`load-baseline`	22 oct	Colapso a los 844 s	1,256	44	3,301 ms	4,290 ms
`load-memory-state`	23 oct	Completó	18,030	1,747	618 ms	2,521 ms
`load-incremental-state`	23 oct	Completó	14,176	1,360	582 ms	2,136 ms
`load-delta-state`	23 oct	Completó	18,314	1,771	698 ms	1,650 ms
`load-store-compaction`	23 oct	Completó	24,878	2,431	40 ms	71 ms

Valores de la última ventana de cada corrida. La última medición registró 29,184 acciones sin errores, con p50 de 26 ms y p95 de 64 ms.

> **Límite.** Estas mediciones se hicieron el 22 y 23 de octubre sobre el almacén de laboratorio SQLite, antes del cambio a PostgreSQL del 27 de octubre. No hay una medición equivalente sobre PostgreSQL ni en hardware real.

13. Estado real y riesgos

Fuente: `docs/ESTADO-COMERCIAL.md` y `docs/PLAN-AUDITORIA-0.3.0.md`, contrastados con el código al 31 de octubre de 2026.

Plataforma	Implementado	Probado	Validado en real
Servidor CulinaryOS	Sí	Parcial: instalador y unidades simulados	No
Cliente Windows (Tauri)	Sí	Sí	No
Cliente Linux en kiosco	Sí	Parcial	No
Android	Compila	No	No
iOS	Sólo código	No	No
Central multiempresa	Sí	Sí, mTLS local	No
Licencias (Worker y Admin)	Sí	Sí	No: intermedias sin desplegar

Auditoría 0.3.0

El plan registra el estado de cada hallazgo al 30 de octubre de 2026. Son 18 hallazgos en tres fases: bloqueadores, resiliencia y validación real.

#	Hallazgo	Estado en el plan
1	Llave raíz fuera de línea	Código hecho, falta el despliegue
2	Outbox sin descarte	Hecho
3	Restauración en otro equipo	Código hecho, falta el simulacro
4	Pagos inciertos	Código hecho, salvo el reverso de un cobro ya aprobado
5	Licencia sin dejar de vender	Código hecho, falta desplegar el Worker
6	Certificados que se renuevan solos	Código hecho, falta probarlo en systemd
7	Transferencia de licencia	Código hecho, falta desplegar y probar en TPM real
8	Puntos de control externos	Código hecho, faltan buckets y prueba contra R2 real
9	Servidor único	Código hecho, faltan simulacro y buckets
10	Inventario concurrente	Hecho
11	Política monetaria y fiscal	Código hecho, faltan nota de crédito y revisión contable
12	Separación de funciones del gerente	Hecho
13	Fallos en cada frontera	Hecho, salvo la caída real de PostgreSQL
14	Versiones y migraciones	Hecho, salvo la prueba con binarios 0.2.x reales
15	Hardware e integraciones reales	Fuera del código: requiere equipos y cuentas
16	Evidencia de pruebas	Escrito, sin ejecutar
17	Estado comercial	Hecho
18	Privacidad	Lo técnico hecho, falta asesoría legal

Antes de vender

1. Simulacro completo en VM y en equipo real, midiendo RTO y RPO.
2. Sandbox de Facturama y de la terminal elegida.
3. Impresora y cajón físicos.
4. Desplegar el Worker con intermedias y sacar la raíz de línea.

5. Crear los buckets R2 de puntos de control y copias.
6. Revisión legal del aviso de privacidad y contable de la política monetaria.
7. Piloto controlado con dinero real.

Riesgos funcionales abiertos

Riesgo	Impacto	Respuesta actual
Servidor único por sucursal	Sin servidor, la operación se detiene	UPS, restauración con objetivo de 4 horas
Nota de crédito sin emitir	Una devolución con CFDI queda `creditNote: pending`	Requiere CFDI de egreso; depende de asesoría contable
IVA del envío	Sale con tasa 0	Confirmar tasa con contador
Purga a 90 días	Reservaciones, llamadas y mensajes no se purgan solos	Sin implementar
Disco sin cifrar	Disco y anillo juntos permiten descifrar	LUKS pendiente de validar
Carga sin medir en PostgreSQL	Capacidad real desconocida	Repetir `Test-Load.ps1`
TPM sin atestación	El Worker confía en la llave declarada	Decisión registrada
Una sola CA por sucursal	Rotarla exige tocar cada terminal	Procedimiento del año 9

Discrepancias encontradas al revisar

Se detectaron al preparar este documento y se corrigieron el 31 de octubre de 2026. Los cambios siguen sin confirmar en cada repositorio. Las cifras de líneas de este documento corresponden a los commits, antes de esos cambios.

Tema	Hallazgo	Estado
Versiones	`Directory.Build.props`, Android, iOS, scripts de empaquetado y pantallas Avalonia decían 0.2.0. `Cargo.toml` de Culinary Admin decía 0.1.0.	Corregido: 0.3.0 y 0.1.3. Android e iOS suben a versionCode 4
Versión de CulinaryOS	`VERSION` dice 0.2.0-dev y `appliance.json` 0.3.0-dev.	Sin cambio: es intencional. La primera es el perfil genérico y la segunda la imagen integrada
Texto dañado	`main.rs` y `docs/CULINARYOS-INSTALACION.md` tenían acentos mal codificados.	Corregido
Llaves	`CulinaryServer/README.md`, `docs/API.md` y `CulinaryAdmin/README.md` describían la raíz en el Worker.	Corregido
Gracia	`CulinaryAdmin/PRODUCT.md` decía 3 días.	Corregido: 7 días
Cifras de pruebas	`README.md` citaba 303 comprobaciones. `docs/ESTADO.md` es un registro del 16 de octubre.	Corregido: 976, y `ESTADO.md` marcado como histórico
SDK	`README.md` y `PRUEBAS.md` decían 10.0.301.	Corregido: 10.0.402
Informe de evidencia	`evidence/INFORME.md` correspondía a `3a24e9b`.	Regenerado sobre `d8fc1ca`, con cambios sin confirmar
Ramas locales	`main` local estaba atrasada en Culinary Restaurant y CulinaryOS.	Corregido
Archivos temporales	Ocho `*.Backup.tmp` en Android, cuatro de ellos versionados, y dos `packages.lock.json` alterados.	Corregido e ignorado. El de Android se conserva porque es correcto
CI	Admin, Server y CulinaryOS no tenían CI, y Culinary Restaurant no corría en push a `main`.	Flujos añadidos. Sin probar hasta subirlos
Commit sin subir	`d8fc1ca` sólo está en local.	Pendiente: requiere autorización
Llaves de prueba	`tests/keys` contiene privadas de prueba. Su pública no coincide con la de producción.	Sin cambio

Anexo A. Archivos y líneas

Todos los archivos fuente de la rama de referencia. Columnas: líneas totales y líneas de código sin blancos ni comentarios.

Culinary Restaurant, rama postgresql

269 archivos · 23,099 líneas de código

Archivo	Líneas	Código
(raíz)	**81**	**74**
.editorconfig	24	22
.gitattributes	11	9
.gitignore	26	23
Directory.Build.props	1	1
MesaNexo.slnx	19	19
.github/workflows	**60**	**57**
ci.yml	60	57
deploy	**175**	**167**
Build-Hybrid.ps1	40	38
install-runtime-extension.py	103	97
mesanexo-central.service	22	22
Start-Hybrid.ps1	10	10
deploy/culinaryos	**1,401**	**1,235**
build-client.sh	16	14
hardware-identity.py	72	60
install.py	441	397
install.sh	38	32
kiosk/activate.sh	9	8
kiosk/install-kiosk.py	271	238
kiosk/openbox.xml	24	23
kiosk/session.py	293	251
kiosk/session.sh	10	8
kiosk/verify-kiosk.py	171	152
renew-tls.py	56	52
runtime	**482**	**433**
export_store.py	61	55
lab_host.py	41	40
lab_probe.py	98	88
mesanexo_ipc.py	282	250
src/MesaNexo.Android	**79**	**71**
AndroidDiscovery.cs	29	26
AndroidVault.cs	26	23
Application.cs	11	10
MainActivity.cs	8	7
MesaNexo.Android.csproj	1	1
Properties/AndroidManifest.xml	1	1
Resources/mipmap-anydpi-v26/ic_launcher.xml	1	1
Resources/values/colors.xml	1	1
Resources/values/styles.xml	1	1
src/MesaNexo.Api	**3,839**	**2,930**
ApiHost.cs	189	139
AuditChain.cs	49	37
BranchService.cs	326	248
CentralSyncWorker.cs	87	67
Checkpoints.cs	206	153
CloudInbox.cs	44	35
CloudPull.cs	37	33
CulinaryServer.cs	72	56
Envelope.cs	58	45
EscPos.cs	214	167
FirstRunSetup.cs	32	25
FiscalProcessing.cs	138	111
FiscalService.cs	43	30
Import.cs	45	40
InventoryWorker.cs	55	42
LabPostgres.cs	131	93
LicenseKeys.cs	125	101
Licenses.cs	175	128
LicenseWorker.cs	137	99
LiveAggregators.cs	47	33
LiveHub.cs	11	9
LiveIntegrations.cs	166	121

Archivo	Líneas	Código
src/MesaNexo.Desktop	**83**	**67**
DesktopVault.cs	49	39
LinuxDiscovery.cs	18	15
MesaNexo.Desktop.csproj	1	1
Program.cs	15	12
src/MesaNexo.Integrations	**659**	**525**
DeliverectAdapter.cs	67	50
ExternalAdapters.cs	114	94
MapsAdapters.cs	38	32
MesaNexo.Integrations.csproj	3	3
MessagingAdapters.cs	63	50
PaymentAdapters.cs	104	88
PmsAdapters.cs	134	104
TerminalAdapters.cs	136	104
src/MesaNexo.Public	**91**	**79**
MesaNexo.Public.csproj	1	1
Program.cs	65	54
wwwroot/app.css	4	4
wwwroot/app.js	18	17
wwwroot/index.html	1	1
wwwroot/sw.js	2	2
src/MesaNexo.Transport	**1**	**1**
MesaNexo.Transport.csproj	1	1
src/MesaNexo.Web	**5,943**	**5,155**
cliente.html	1	1
index.html	1	1
playwright.config.ts	4	3
scripts/prepare-bridge.mjs	12	12
shared/product-options.ts	12	6
src-tauri/build.rs	6	5
src-tauri/Cargo.toml	10	10
src-tauri/src/kiosk.rs	370	324
src-tauri/src/main.rs	70	69
src/backoffice.tsx	314	271
src/battery-status.tsx	37	28
src/bridge.ts	45	36
src/button-motion.ts	18	13
src/client-update.tsx	16	14
src/context-menu.tsx	112	70
src/counter-page.tsx	96	85
src/customer-display.ts	17	13
src/customer-main.tsx	28	24
src/ecosystem.tsx	272	238
src/form-dialog.tsx	146	115
src/integrations.tsx	255	221
src/kiosk-presentation.ts	30	20
src/labels.ts	107	107
src/license-lock.tsx	41	35
src/loyalty.tsx	185	147
src/main.tsx	341	304
src/motion.tsx	159	118
src/network-menu.tsx	78	66
src/order-product-dialog.tsx	73	61
src/overview-page.tsx	54	51
src/power-menu.tsx	49	39
src/preparation-notes.tsx	10	7
src/profile-menu.tsx	38	31
src/secret-input.tsx	9	6
src/service-board.tsx	118	107
src/service-priorities.ts	39	29
src/setup-wizard.tsx	57	45
src/split-dialog.tsx	32	27
src/studio-navigation.tsx	44	38

Archivo	Líneas	Código
LiveMapsAndCalls.cs	70	50
LiveMessaging.cs	63	46
LivePayments.cs	52	40
LivePms.cs	74	55
LiveTerminals.cs	99	77
MesaNexo.Api.csproj	1	1
Migration.cs	21	18
OffsiteBackup.cs	80	58
OutboxWorker.cs	14	9
PasswordReset.cs	25	18
Passwords.cs	24	19
PostgresStore.cs	500	383
PrintDriver.cs	107	87
Program.cs	60	44
Reports.cs	27	24
SchemaUpgrade.cs	23	20
StateBases.cs	29	22
Storage.cs	69	53
StoreMigration.cs	58	47
Telephony.cs	56	47
src/MesaNexo.Bridge	**248**	**199**
MesaNexo.Bridge.csproj	1	1
Program.cs	247	198
src/MesaNexo.Central	**739**	**601**
CentralService.cs	34	26
CentralStore.cs	106	80
CommerceService.cs	150	120
DeliveryHub.cs	20	17
MesaNexo.Central.csproj	1	1
PaymentReceipts.cs	48	40
Program.cs	83	71
PublicInvoice.cs	37	30
TenantEndpoints.cs	30	25
TenantStore.cs	230	191
src/MesaNexo.Client	**1,979**	**1,695**
Administration.cs	73	60
App.cs	26	21
Brand.cs	15	14
Connection.cs	175	144
Discovery.cs	4	3
EcosystemPages.cs	138	110
Forms.cs	119	100
LicenseScreen.cs	74	63
LiveConnection.cs	41	32
MainView.cs	117	99
MesaNexo.Client.csproj	1	1
ProductPicture.cs	41	32
PrototypeAccount.cs	139	124
PrototypeAdministration.cs	82	67
PrototypeLabels.cs	6	5
PrototypeMotion.cs	148	115
PrototypeProductDialog.cs	88	83
PrototypeService.cs	302	273
PrototypeShell.cs	262	239
PrototypeTheme.axaml	32	32
UpdateDialog.cs	36	29
Updates.cs	60	49
src/MesaNexo.Core	**3,900**	**3,181**
Backoffice.cs	397	336
BlindCash.cs	71	62
Cents.cs	26	18
Checkpoints.cs	73	52
Closings.cs	86	75
CloudProposals.cs	48	41
Contract.cs	10	6
Delivery.cs	183	153
Fiscal.cs	70	57
Frontoffice.cs	169	131
IntegrationContracts.cs	10	9
Integrations.cs	492	412
Inventory.cs	300	240
Json.cs	182	146
Licensing.cs	169	127
src/support-page.tsx	32	30
src/theme.css	1,261	1,170
src/topbar-fit.ts	18	12
src/ui.tsx	100	87
src/visit-label.ts	13	10
tests/hybrid.spec.ts	736	673
tests/kiosk.spec.ts	223	168
tests/native-startup.mjs	32	32
tests/setup.spec.ts	86	73
tests/topbar.spec.ts	131	99
vite.config.ts	5	4
src/MesaNexo.iOS	**59**	**49**
AppDelegate.cs	10	9
IosDiscovery.cs	32	25
IosVault.cs	12	11
Main.cs	4	3
MesaNexo.iOS.csproj	1	1
tests	**1,201**	**1,152**
Invoke-LabSmoke.ps1	4	4
test_culinaryos_install.py	527	521
test_culinaryos_kiosk.py	240	215
test_lab_launcher.py	124	111
test_runtime_extension.py	306	301
tests/MesaNexo.Hybrid.Tests	**153**	**138**
MesaNexo.Hybrid.Tests.csproj	1	1
Program.cs	152	137
tests/MesaNexo.Load	**305**	**252**
MesaNexo.Load.csproj	1	1
Program.cs	304	251
tests/MesaNexo.Tests	**4,135**	**3,640**
AdapterTests.cs	48	45
ApprovalTests.cs	128	116
BackofficeTests.cs	149	126
BoundaryTests.cs	76	64
BusinessTests.cs	86	83
ButtonFeedbackTests.cs	118	109
CentralTests.cs	106	95
CheckpointTests.cs	176	158
CloudScenario.cs	113	105
DevMapTests.cs	177	169
FiscalTests.cs	193	157
FrontofficeTests.cs	107	94
IntegrationTests.cs	204	179
InventoryTests.cs	115	104
LicenseTests.cs	413	363
LiveIntegrationTests.cs	338	287
LoyaltyTests.cs	145	128
MesaNexo.Tests.csproj	1	1
MigrationTests.cs	96	93
MoneyTests.cs	123	106
PrintTests.cs	184	149
PrivacyTests.cs	62	59
Program.cs	241	225
SchemaTests.cs	43	39
ServerSmoke.cs	30	25
SetupTests.cs	72	61
StoreTests.cs	256	209
TenantTests.cs	105	101
TestLicense.cs	81	61
UiTests.cs	149	129
tests/fixtures	**520**	**474**
delivered-extensions/r1-r2-crlf.py	114	107
delivered-extensions/r1-r2-lf.py	114	107
delivered-extensions/r13-r14.py	292	260
tools	**971**	**924**
build.ps1	23	23
evidence_report.py	62	49
fetch_update.py	72	71
Get-Postgres.ps1	15	14
migrate_prototype.py	138	133
package_deb.py	71	69
package_delivery.py	86	80
package_windows.ps1	49	48

Archivo	Líneas	Código
LiveIntegrations.cs	249	185
Loyalty.cs	193	166
MesaNexo.Core.csproj	8	6
PaymentStates.cs	38	30
PosExtensions.cs	206	165
Privacy.cs	47	40
Refunds.cs	65	51
Restaurant.cs	618	526
StateDelta.cs	114	86
TableQr.cs	76	61

Archivo	Líneas	Código
Prepare-Branding.ps1	55	53
Prepare-CulinaryOsBundle.py	65	64
Prepare-CulinaryOsIso.ps1	40	40
Prepare-CulinaryOsKiosk.py	51	44
Start-Lab.ps1	134	128
Test-Hybrid.ps1	16	16
Test-Load.ps1	21	21
Test.ps1	16	16
Use-RootKey.ps1	14	12
Verify-Branding.ps1	43	43

CulinaryOS, rama postgresql

85 archivos · 5,059 líneas de código

Archivo	Líneas	Código
(raíz)	**20**	**19**
.gitattributes	8	8
.gitignore	12	11
config	**22**	**21**
appliance-preseed.cfg	22	21
config/appliance-bootloaders	**85**	**81**
grub-efi/grub.cfg	21	20
grub-efi/theme.cfg	10	10
grub-pc/grub.cfg	21	20
grub-pc/theme.cfg	10	10
syslinux_common/install_gui.cfg	18	16
syslinux_common/menu.cfg	5	5
os/appliance	**476**	**398**
…ghtdm.conf.d/95-culinary-restaurant-kiosk.conf	2	2
…c/polkit-1/rules.d/50-culinaryos-network.rules	15	15
etc/polkit-1/rules.d/50-culinaryos-power.rules	13	13
…m/culinaryos-backend.service.d/restaurant.conf	3	3
etc/systemd/system/culinaryos-data.service	45	41
…/culinaryos-firstboot.service.d/appliance.conf	5	4
usr/local/lib/culinaryos/appliance.py	174	143
usr/local/lib/culinaryos/recovery.py	219	177
os/overlay	**2,605**	**2,431**
etc/avahi/avahi-daemon.conf	21	17
etc/chrony/chrony.conf	10	9
etc/dbus-1/system.d/90-culinaryos-avahi.conf	13	13
etc/default/grub.d/60-culinaryos-splash.cfg	23	20
etc/lightdm/lightdm-gtk-greeter.conf	9	9
…NetworkManager/conf.d/20-culinaryos-local.conf	3	2
etc/nftables.conf	22	19
etc/nut/nut.conf	2	1
etc/postgresql/17/main/conf.d/culinaryos.conf	14	12
etc/sysctl.d/60-culinaryos.conf	10	10
etc/systemd/journald.conf.d/culinaryos.conf	9	9
etc/systemd/system/culinaryos-backend.service	45	43
etc/systemd/system/culinaryos-data.service	43	41
…tc/systemd/system/culinaryos-firstboot.service	23	21
etc/systemd/system/culinaryos-health.service	35	32
etc/systemd/system/culinaryos-health.timer	11	9
…/systemd/system/culinaryos-maintenance.service	12	11
etc/systemd/system/culinaryos-publish.service	29	27
etc/systemd/system/culinaryos-rekey.service	26	25
etc/systemd/system/culinaryos-snapshot.service	25	24
etc/systemd/system/culinaryos-sync.service	36	35
etc/systemd/system/culinaryos-sync.timer	11	9
…stem/display-manager.service.d/culinaryos.conf	6	6
…ty@tty1.service.d/20-culinaryos-graphical.conf	4	2

Archivo	Líneas	Código
…md/system/getty@tty1.service.d/culinaryos.conf	4	3
…temd/system/nftables.service.d/culinaryos.conf	2	2
usr/local/lib/culinaryos/apphost.py	118	109
usr/local/lib/culinaryos/backup.py	149	137
usr/local/lib/culinaryos/bootconfig.py	57	52
usr/local/lib/culinaryos/branding.py	54	48
usr/local/lib/culinaryos/dataio.py	128	117
usr/local/lib/culinaryos/deployment.py	120	112
usr/local/lib/culinaryos/discovery.py	52	46
usr/local/lib/culinaryos/health.py	232	221
usr/local/lib/culinaryos/lan.py	147	139
usr/local/lib/culinaryos/pki.py	173	156
usr/local/lib/culinaryos/provision.py	222	208
usr/local/lib/culinaryos/runtimehealth.py	59	57
usr/local/lib/culinaryos/securedata.py	112	105
usr/local/lib/culinaryos/site.py	169	160
usr/local/lib/culinaryos/store.py	160	158
usr/local/lib/culinaryos/syncworker.py	46	44
usr/local/lib/culinaryos/ups.py	98	92
usr/local/lib/culinaryos/welcome.py	61	59
scripts	**722**	**689**
audit-appliance.py	62	61
build-appliance.sh	116	112
build.sh	69	65
check-systemd-credential.py	8	8
install-lab.sh	62	58
prepare-appliance-input.py	49	46
repair-lab-data.sh	70	67
repair-lab-network.sh	69	65
restore-lab-boot.sh	20	17
run-vm.sh	34	33
update-lab-boot.sh	106	101
verify-linux.sh	16	15
verify.ps1	41	41
tests	**1,481**	**1,420**
test_appliance.py	238	218
test_backup.py	72	71
test_data.py	273	261
test_diagnostics.py	100	99
test_firstboot.py	75	65
test_network.py	124	124
test_pki.py	110	108
test_project.py	111	107
test_recovery.py	143	137
test_runtime.py	142	138
test_sync_tls.py	93	92

Culinary Admin, rama master

21 archivos · 1,191 líneas de código

Archivo	Líneas	Código
(raíz)	**34**	**32**
.gitignore	9	9
index.html	13	13

Archivo	Líneas	Código
theme.css	244	229
ui.tsx	84	52
src-tauri	**14**	**14**

Archivo	Líneas	Código
playwright.config.ts	8	7
vite.config.ts	4	3
src	**1,190**	**832**
api.ts	207	137
App.tsx	60	40
gate.tsx	138	80
licenses.tsx	238	153
main.tsx	9	8
releases.tsx	105	66
support.tsx	43	28
team.tsx	62	39

Archivo	Líneas	Código
build.rs	3	3
Cargo.toml	11	11
src-tauri/src	**9**	**8**
lib.rs	5	4
main.rs	4	4
tests	**269**	**241**
panel.spec.ts	269	241
tools	**68**	**64**
build-android.ps1	43	40
build-windows.ps1	25	24

Culinary Server, rama main

34 archivos · 2,416 líneas de código

Archivo	Líneas	Código
(raíz)	**51**	**46**
.gitattributes	1	1
.gitignore	4	4
vitest.config.ts	46	41
migrations	**160**	**120**
0001_inicial.sql	153	115
0002_nivel_y_addons.sql	2	2
0003_licencia_eliminada.sql	2	1
0004_llave_liberada.sql	3	2
scripts	**127**	**93**
generate-secrets.mjs	39	28
sign-intermediate.mjs	47	38
sign-key-floor.mjs	15	8
sign-release.mjs	26	19
scripts/lib	**42**	**30**
jws.mjs	42	30
src	**2,162**	**1,371**
admin-auth.ts	252	160
admin.ts	316	209
branch.ts	178	113

Archivo	Líneas	Código
catalog.ts	59	39
crypto.ts	235	138
db.ts	72	44
http.ts	131	77
index.ts	55	37
keys.ts	151	90
licenses.ts	174	134
modules.d.ts	6	4
signature.ts	88	54
tickets.ts	120	77
updates.ts	325	195
test	**878**	**756**
admin.spec.ts	164	144
env.d.ts	16	16
helpers.ts	136	109
keys.spec.ts	44	38
licenses.spec.ts	285	255
setup.ts	13	10
support.spec.ts	68	58
updates.spec.ts	152	126

Anexo B. Rutas de API

Backend de sucursal, MesaNexo.Api

Método	Ruta	Función
GET	`/api/identity`	Sitio y contrato de API, antes de enviar credenciales
GET	`/api/health`	Estado del servicio
POST	`/api/login`	Sesión, con límite de intentos
GET	`/api/license`	Estado de licencia, disponible antes de entrar
POST	`/api/license/activate`	Activar o reactivar
GET, POST	`/api/setup`	Asistente de primera instalación
POST	`/api/updates/check`	Consultar versión
POST	`/api/updates/download`	Descargar paquete
POST	`/api/device/can-install`	Comprobar condiciones para instalar
GET	`/api/preview/{id}`	Vista previa de impresión
GET	`/api/reports`	Reportes históricos paginados
GET	`/api/audit/verify`	Verificar cadena de auditoría
GET	`/api/state`	Estado completo o delta
POST	`/api/action`	Toda mutación del dominio
GET	`/api/operations/{id}`	Recibo por UUID
Hub	`/live`	SignalR de revisión

MesaNexo.Central y MesaNexo.Public

Servicio	Método	Ruta
Central	GET	`/api/identity`, `/api/state`
Central	POST	`/api/login`, `/api/action`, `/v1/events`, `/v1/cloud/ack`
Central	GET	`/v1/cloud/pending`
Public	GET	`/public/{slug}/manifest`, `/public/{slug}/catalog`, `/public/{slug}/orders/{id}`, `/public/{slug}/table/{token}`, `/s/{slug}`, `/invoice`
Public	POST	`/public/{slug}/orders`, `/public/{slug}/table/{token}`, `/public/invoice`, `/webhooks/deliverect/{slug}`

Culinary Server

Método	Ruta	Método	Ruta
POST	`/branch/v1/challenges`	GET	`/admin/v1/audit`
POST	`/branch/v1/activate`	GET, POST	`/admin/v1/licenses`
POST	`/branch/v1/license/renew`	GET, PATCH, DELETE	`/admin/v1/licenses/:id`
POST	`/branch/v1/updates/check`	POST	`/admin/v1/licenses/:id/renew`
GET	`/branch/v1/downloads/:releaseId`	POST	`/admin/v1/licenses/:id/revoke`
GET, POST	`/branch/v1/tickets`	POST	`/admin/v1/licenses/:id/reset-activation`
POST	`/branch/v1/tickets/:id/messages`	GET	`/admin/v1/registry`
PUT	`/branch/v1/registry`	GET, POST	`/admin/v1/releases`
POST	`/admin/v1/challenges`	DELETE	`/admin/v1/releases/:id`
POST	`/admin/v1/enroll`	PUT	`/admin/v1/releases/:id/parts/:part`
POST	`/admin/v1/enroll/confirm`	POST	`/admin/v1/releases/:id/complete`
POST	`/admin/v1/login`	PUT	`/admin/v1/releases/:id/manifest`
POST	`/admin/v1/logout`	GET, POST	`/admin/v1/publications`
POST	`/admin/v1/password`	POST	`/admin/v1/publications/:id/withdraw`
GET	`/admin/v1/me`	GET	`/admin/v1/tickets`
POST	`/admin/v1/invites`	GET	`/admin/v1/tickets/:id`
GET	`/admin/v1/admins`	POST	`/admin/v1/tickets/:id/messages`
DELETE	`/admin/v1/admins/:id`	POST	`/admin/v1/tickets/:id/close`
GET	`/admin/v1/root-key`		

43 manejadores en total, 9 de sucursal y 34 de panel. Contrato completo en `CulinaryServer/docs/API.md`.

Anexo C. Acciones del dominio

Las 122 acciones de `Restaurant` que acepta `POST /api/action`, extraídas de `src/MesaNexo.Core`.

Familia	Acciones	Familia	Acciones
`visit`	close, customer, deliverAll, divide, issueBill, merge, mergeMany, move, open, pricelist, quick, reassignZone, reopen, split, transfer, undivide, unlock	`call`	attend
`cash`	close, limit, movement, open, withdraw	`callerid`	simulate
`delivery`	assign, create, dispatch, pickup, reconcile	`command`	send
`inventory`	configure, ingredient, levels, variance, warehouse	`courier`	save
`stock`	count, process, receive, return, waste	`device`	enrollment
`aggregator`	decide, handoff, simulate, sync	`drawer`	request
`purchase`	cancel, order, receive, suggest	`geo`	lookup
`terminal`	cancel, charge, manual, reconcile	`installation`	authorize
`catalog`	category, product, toggle	`integration`	configure
`customer`	erase, export, save	`loyalty`	configure
`paylink`	cancel, create, simulate	`message`	test
`print`	claim, resolve, retry	`notification`	attend
`report`	abc, x, z	`pac`	stamp
`staff`	configure, meal, settle	`payable`	pay
`table`	clean, qr, save	`payment`	add
`transfer`	cancel, create, receive	`pms`	charge
`wallet`	adjust, pay, topup	`pricelist`	save
`commerce`	configure, decide	`promotion`	save
`expense`	account, record	`reservation`	save
`petty`	configure, fund	`review`	simulate
`qr`	configure, decide	`sale`	quick
`recipe`	bind, save	`session`	revoke
`request`	create, decide	`settings`	save
`ticket`	create, reply	`station`	configure
`user`	pin, save	`supplier`	save
`address`	save	`unit`	state
`authorize`	pin	`zone`	save
`branch`	save		

Anexo D. Glosario

Término	Significado
ACK	Acuse que confirma que el receptor guardó un evento
CAS	Escritura que sólo procede si la versión no cambió
CFDI	Comprobante fiscal digital en México
JWS	Documento JSON firmado
KDS	Pantalla de cocina
mTLS	TLS con certificado de ambos lados
Outbox	Tabla de eventos pendientes de enviar
PAC	Proveedor autorizado de timbrado
PEPS	Primeras entradas, primeras salidas
PMS	Sistema de gestión hotelera
R2 y D1	Almacén de objetos y base SQL de Cloudflare
RPO y RTO	Datos que se pueden perder y tiempo para volver a operar
Sidecar	Proceso auxiliar que acompaña a la aplicación
TOTP	Código temporal de seis dígitos
TPM	Chip que guarda llaves sin permitir exportarlas